

Situación de aprendizaje 1.3: El código de tu economía

Tecnologías de la Información y la Comunicación II

2º de Bachillerato

Situación de aprendizaje

Bloque 1: Desarrollo de Software Situación de aprendizaje 3: El Código de tu economía



Imagen de elaboración propia generada con Gemini. El código de tu economía (CC BY-NC-ND)

1. Inicializa tu aprendizaje

1. Introducción

Como ya has visto en la situación de aprendizaje anterior, el software forma parte de nuestro día a día. Cada vez que sacas el móvil para consultar el saldo de tu cuenta, reservas una cita médica desde una app o recibes una notificación de que tu pedido ya está en camino, hay alguien —o algo— detrás que ha tomado decisiones: si el saldo es suficiente, muestra el mensaje; si no, avisa al usuario. Ese "alguien" es un programa informático, y esa lógica la escribió una persona como tú.

Programar no es magia ni está reservado a genios; es, sencillamente, aprender a pensar de forma lógica para resolver problemas y traducir esas soluciones a un idioma que un ordenador pueda entender.

En esta situación de aprendizaje descubrirás cómo se construyen esas aplicaciones desde dentro, aprendiendo a crear programas capaces de almacenar datos, tomar decisiones, repetir acciones o reutilizar código para hacer soluciones más organizadas y eficientes. A medida que avances, comprobarás que programar no consiste solo en escribir código, sino en desarrollar una forma de pensar lógica y creativa que resulta útil tanto en el ámbito tecnológico como en muchas situaciones de la vida cotidiana y profesional.

2. Todo un reto por delante

¿Te has parado a pensar la cantidad de tiempo que perdemos a final de mes intentando cuadrar facturas, gastos imprevistos y ahorros? Gestionar la economía doméstica puede convertirse en un auténtico dolor de cabeza, pero también es el escenario perfecto para que la tecnología trabaje a tu favor.

En esta situación de aprendizaje, tu reto será programar desde cero tu propio Gestor de Gastos Personales. A lo largo de la unidad aprenderás a almacenar datos, programar alertas automáticas de presupuesto y guardar historiales de forma organizada para construir esta aplicación paso a paso (puedes consultar todos los detalles en la página del Reto: Toma el control de tus finanzas).

¡Anímate a dar el paso! Verás lo satisfactorio que resulta resolver un problema cotidiano escribiendo tu propio código.



Imagen de creación propia generada con Gemini
<<https://gemini.google.com/>> . Toma el control de
tus finanzas (CC BY-NC-SA
<<http://creativecommons.org/licenses/>>)

3. Estos serán tus logros

A lo largo de esta situación de aprendizaje, irás adquiriendo las piezas necesarias para construir tu propio Gestor de Gastos y entender cómo funciona el software por dentro. Al finalizar, habrás conseguido lo siguiente:

- Conocerás qué es un programa informático, cómo se estructura internamente y qué tipos de lenguajes de programación existen.
- Identificarás los elementos básicos de un programa: variables, constantes, tipos de datos, operadores y expresiones.
- Aprenderás a comunicarte con el usuario a través de la entrada y salida de datos, y a documentar tu código con comentarios.
- Identificarás cuándo y cómo utilizar estructuras de control para tomar decisiones y repetir acciones dentro de un programa.
- Descubrirás diferentes formas de organizar la información mediante estructuras de datos.
- Crearás funciones para reutilizar código y construir programas más ordenados y modulares y harás uso de bibliotecas.
- Serás capaz de leer y escribir archivos para que tus programas guarden y recuperen información de forma persistente.
- Identificarás los conceptos básicos de la Programación Orientada a Objetos (clases y objetos), una técnica que te permitirá diseñar programas mediante la representación de elementos del mundo real, (por ejemplo, creando un objeto "Cuenta Bancaria").

4. Conexión con la vida real.

Puede que en algún momento te hayas preguntado: ¿para qué voy a aprender a programar si no quiero ser informático? La respuesta es más sencilla de lo que parece.

Vivimos rodeados de software: las apps que usamos para pagar, para organizarnos o para comunicarnos han sido creadas por personas que aprendieron exactamente lo que tú vas a aprender en esta situación de aprendizaje. Pero más allá de crear aplicaciones, programar desarrolla una forma de pensar lógica y estructurada que resulta útil en cualquier ámbito profesional: desde la gestión de un negocio hasta la sanidad, pasando por la educación o la administración pública.

Más allá de la informática pura, desarrollar esta habilidad también tiene un impacto directo y muy positivo en tu día a día:

- Resolución práctica de problemas:** aprenderás a enfrentarte a un obstáculo complejo dividiéndolo en pequeños pasos lógicos y manejables. El divide y vencerás que viste en la situación de aprendizaje 1.
- Pensamiento crítico:** entenderás de verdad cómo las aplicaciones que usas a diario procesan tus datos y por qué toman ciertas decisiones.

<https://www.youtube.com/embed/Jcq9dgQqgS4>
<<https://www.youtube.com/embed/Jcq9dgQqgS4>>

<<https://www.youtube.com/embed/Jcq9dgQqgS4>>

Error de
configuración
del reproductor
de vídeo

Error 153

Ver vídeo en YouTube

Más información

Error 153

Video de CEIVA <<https://www.youtube.com/@ceivacentrodeeducacionin9111>> . Importancia
de aprender a programar y el Pensamiento Computacional <<https://www.youtube.com/watch?v=Jcq9dgQqgS4>> (Licencia de YouTube estándar <<https://www.youtube.com/static?template=terms>>)

5. Mapa conceptual

Descarga [aquí](#) la versión PDF.

Haz clic sobre la imagen para ampliarla

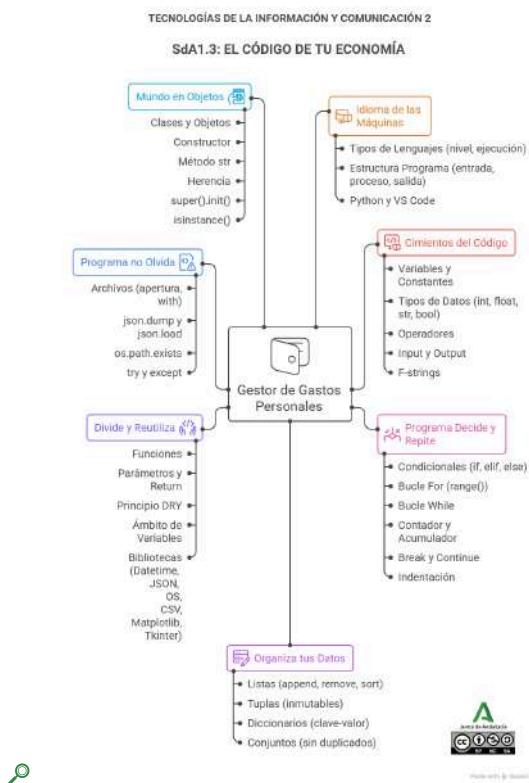


Imagen de elaboración propia generada con Napkin AI <<https://www.napkin.ai/es/>> .
Mapa conceptual SdA1.3 (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

6. Estructura de este recurso

Esta situación de aprendizaje está organizada en distintos bloques de contenidos y actividades que te ayudarán a comprender el tema, y un Reto Final para poner en práctica lo aprendido.

La estructura es la siguiente:

- **Menú lateral izquierdo**, donde puedes acceder a cada uno de los apartados (puedes ocultarlo pulsando la "X", en la parte superior izquierda de la página).
- **Descripción del Reto Final** e información inicial, para que puedas hacerte una idea general de por qué y para qué vas a trabajar en esta situación de aprendizaje.
- **Actividades de recuerdo**, para activar los conocimientos previos.
- **Mini reto**, para poner en funcionamiento dichos conocimientos y prepararte para el reto final.
- **Bloques de contenido** con contenidos destacados, imágenes y mapas interactivos, actividades relacionadas y conceptos clave... La importancia de estos contenidos se identifican con **íconos**.
- **Resumen de contenidos**.
- **Documento para descargar**.

En esta imagen se describen los principales íconos y su significado:

Haz clic en la imagen para ampliarla

ICONOS

Y

SIGNIFICADOS

Objetivos que se quieren alcanzar con la situación de aprendizaje

Lectura de textos
Contenido complementario

Imagen o Mapa
Contenido visual

Orientaciones e instrucciones de temas y ejercicios

Multimedia (vídeos, audios...) y complementario

Conceptos Clave
Contenido de estudio

Descripción de Retos
Contenido importante para el Reto Final

Modelo de tarea resuelta

Tarea práctica
Ejercicios de repaso

Saber más
Contenido no obligatorio pero aconsejable

Caso práctico o ejemplo real
Ayuda a comprender el tema

Descripción de actividad
Tareas obligatorias

Importante
Contenido de estudio

Tarea para reflexionar
Ayuda a comprender el tema

Actividad que requiere Internet

Conceptos e ideas destacadas
Contenido de estudio

Nota aclaratoria
Contenido complementario

Tarea escrita obligatoria

Imagen de elaboración propia creada con Canva (2025)
https://www.canva.com/es_es/ . Iconos eXeLearning
 (CC BY-NC-SA <<http://creativecommons.org/licenses/>?lang=es>)

2. Diagnóstico del sistema

1. Conocimientos previos

Antes de comenzar con los nuevos contenidos, vamos a hacer una parada técnica. Estas actividades no son un examen: su objetivo es que recuerdes lo que ya sabes, tomes conciencia de tu punto de partida y establezcas conexiones entre lo que conoces y lo que vas a aprender en esta SdA. No te preocupes si no lo recuerdas todo, lo importante es que vayas evocando aprendizajes anteriores.

Como recordarás, todo desarrollo de software profesional comienza mucho antes de teclear la primera línea de código. Vamos a repasar brevemente el análisis del problema, los entornos de trabajo y la automatización. No te preocupes si dudas en alguna respuesta; el objetivo aquí es identificar de dónde partimos para afrontar el reto con éxito.



Imagen de elaboración propia generada con LeonardoAI <<https://leonardo.ai/>> .
 Conocimientos previos (CC BY-NC-SA <<http://creativecommons.org/licenses/>?lang=es>)

Opción A. Verdadero o falso

Responde todas las preguntas del cuestionario.

La transformación digital consiste únicamente en digitalizar documentos en papel.

☐ Verdadero ☐ Falso

Un algoritmo debe estar escrito obligatoriamente en un lenguaje de programación.

☐ Verdadero ☐ Falso

El enfoque Top-Down consiste en dividir un problema complejo en subproblemas más pequeños y manejables.

☐ Verdadero ☐ Falso

Un IDE es un entorno de desarrollo que integra editor, compilador y depurador en una sola herramienta.

☐ Verdadero ☐ Falso

La fase de depuración del ciclo de vida del software consiste en añadir nuevas funcionalidades al programa.

☐ Verdadero ☐ Falso

El control de versiones permite gestionar los cambios realizados en el código a lo largo del tiempo.

☐ Verdadero ☐ Falso

Teniendo en cuenta el Ciclo de Vida del Software, el primer paso lógico para construir nuestro Gestor de Gastos es abrir el Entorno de Desarrollo Integrado (IDE) y empezar a programar la interfaz gráfica, dejando el análisis matemático para el final.

☐ Verdadero ☐ Falso

Opción B. ¿Cuánto recuerdas?

Elija la respuesta correcta

6  0  0  0

 00:00 



[Pulse aquí para jugar](#)

Opción C. Ordena las fases

Ordena correctamente las fases del desarrollo de un programa, de la primera a la última:

Análisis del problema: ¿qué necesita hacer el programa?

Depuración: corregir los errores encontrados.

Pruebas: ejecutar y comprobar que funciona.

Mantenimiento: mejorar y actualizar el programa.

Diseño de la solución: pseudocódigo y esquema.

Programación: escribir el código en Python.

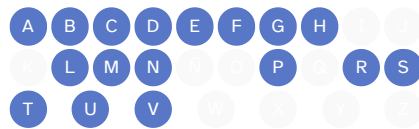
[Comprobar](#)

Opción D. El Rosco del Desarrollo

Observa las letras, identifica y rellena las palabras que faltan.

 0  0

 06:00  



¿Preparado?

[Pulsa aquí para empezar](#)

Escribe la palabra correcta y pulsa en Responder. Si dudas, pulsa en Seguir.



3. Antes de programar ... ¡piensa!



1. Este será tu mini reto

Antes de escribir una sola línea de código, cualquier buen programador hace algo fundamental: pensar. Pensar en el problema, en cómo lo resuelve una persona de forma manual y en qué necesitaría saber un programa para hacerlo automáticamente. Eso es exactamente lo que vas a hacer tú ahora.

Parte 1 — Tu forma de gestionar los gastos

Piensa en cómo gestionas actualmente tus gastos personales o familiares, aunque sea de forma aproximada, y responde a estas preguntas:

- ¿Anotas en algún sitio lo que gastas? ¿Dónde y cómo?
- ¿Cómo sabes si te estás pasando de presupuesto?
- ¿Qué pasos sigues, aunque sean mentales, cuando quieres saber cuánto llevas gastado en un mes? Escribe esos pasos en orden, como si le estuvieras explicando el proceso a alguien que no sabe nada de tu economía. No hace falta que sea perfecto: escríbelo como te salga, con tus propias palabras.

Parte 2 — ¿Qué necesitaría saber tu programa?

Ahora imagina que vas a encargarle esa tarea a un programa informático. Para que pueda hacer lo mismo que tú, necesita que le digas exactamente qué información manejar y qué tiene que ser capaz de hacer. Completa lo siguiente:

- ¿Qué datos tendría que guardar? (Por ejemplo: el importe, la fecha, la categoría del gasto...).
- ¿Qué tendría que ser capaz de calcular o comprobar? (Por ejemplo: el total gastado, si se ha superado un límite...).
- ¿Qué tendría que mostrarle al usuario? (Por ejemplo: un resumen mensual, una alerta de presupuesto...).

Cuando termines, relee lo que has escrito. Acabas de hacer lo que ya has aprendido en la situación de aprendizaje anterior y que los programadores llaman análisis del problema: entender qué tiene que resolver el programa antes de ponerse a construirlo. Guarda tus respuestas porque te serán útiles más adelante cuando empieces a construir tu Gestor de Gastos.

Esta actividad no tiene una respuesta única ni correcta. Lo importante es que reflexiones y te enfrentes al problema con lo que ya sabes.



Imagen de elaboración propia generada con LeonardoAI <<https://leonardo.ai/>> . Analizando los datos de tu día a día (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

4. El lenguaje de las máquinas

1. Introducción

Ya tienes claro el reto que te espera y sabes qué vas a aprender. Ahora toca ponerse manos a la obra. Pero antes de escribir una sola línea de código, necesitas entender algo fundamental: **los ordenadores no hablan tu idioma**.

Cuando tú piensas en registrar un gasto, piensas en palabras, números y categorías.

El ordenador, en cambio, solo entiende unos y ceros. Entre tu idea y la máquina existe una distancia enorme, y los **lenguajes de programación** son el puente que la salva.

En este bloque darás tus primeros pasos en ese puente:

- Descubrirás cómo se clasifican los lenguajes de programación, qué diferencia hay entre los que el ordenador entiende directamente y los que necesitan traducción, y cómo se estructura internamente cualquier programa.
- Conocerás Python, el lenguaje que vas a usar para construir tu Gestor de Gastos, y prepararás tu entorno de trabajo con VS Code.

Fijate bien en la imagen: a la izquierda reconoces tu mundo, el de las personas, con conceptos como importe, categoría o fecha flotando en el ambiente de un hogar cotidiano.

A la derecha, el mundo de las máquinas: servidores, circuitos y un torrente de ceros y unos que no significan nada para ti... todavía.

En el centro, el puente: código Python real, con sus def, sus if y sus funciones, tendido entre ambos mundos. Y en ese puente, una persona caminando. Esa persona eres tú. Este bloque es tu primer paso sobre él.

Al terminar este bloque tendrás todo lo necesario para escribir y ejecutar tu primer programa real. ¿Empezamos?

Haz clic sobre la imagen para ampliarla



Imagen de elaboración propia generada con Gemini <<https://gemini.google.com/>> . El puente de la programación (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

2. Revisamos objetivos del bloque

Lee cada afirmación y decide si es verdadera o falsa.

4

00:00



[Pulse aquí para jugar](#)

4.1 Tipos y estructura general

1. De la idea al código real

Como viste en la primera situación de aprendizaje, ya sabes cómo organizar una mudanza o hacer un café paso a paso. Aprendiste a usar el "divide y vencerás" y a plasmar esas soluciones en diagramas de flujo y pseudocódigo. Pero, como bien sabes, un ordenador no entiende nuestro pseudocódigo; necesita que le hablemos en su propio idioma. Aquí es donde entran los lenguajes de programación.

Un **lenguaje de programación** no es más que un conjunto de reglas, símbolos y palabras (casi siempre en inglés) que nos permite darle instrucciones exactas a la máquina.

Tipos de lenguajes de programación

No todos los lenguajes son iguales. Se clasifican principalmente según dos criterios:

1. Según su cercanía a la máquina (Nivel):

- ◆ **Bajo nivel:** hablan casi directamente el "idioma" electrónico del ordenador (ceros y unos). Son difícilísimos de leer para un humano, pero rapidísimos para la máquina.

```
lenguaje_bajo_nivel.asm X
D: > REA > lenguaje_bajo_nivel.asm
1 ; Programa en ensamblador x86
2 ; Suma dos números: 5 + 3
3
4 section .data
5     num1 db 5 ; primer número
6     num2 db 3 ; segundo número
7     resultado db 0 ; almacena el resultado
8
9 section .text
10    global _start
11
12    start:
13    mov al, [num1] ; carga num1 en el registro AL
14    add al, [num2] ; suma num2 al registro AL
15    mov [resultado], al ; guarda el resultado
```

Imagen de elaboración propia. Código lenguaje ensamblador (CC BY-NC-SA
<<http://creativecommons.org/licenses/>>)

- ◆ **Alto nivel:** están diseñados para que tú los entiendas fácilmente. Usan palabras como *print* (imprimir) o *if* (si condicional). La máquina necesita "traducirlos" antes de ejecutarlos. (Nosotros usaremos estos).

```
lenguaje_alto_nivel.py X
D: > REA > lenguaje_alto_nivel.py > ...
1 # Programa en Python
2 # Suma dos números: 5 + 3
3
4 num1 = 5
5 num2 = 3
6 resultado = num1 + num2
7
```

Imagen de elaboración propia. Código lenguaje alto nivel (CC BY-NC-SA
<<http://creativecommons.org/licenses/>>)

2. Según como se traducen (Ejecución):

- ◆ **Compilados:** un programa (compilador) coge todo tu código, lo traduce de golpe al idioma de la máquina y crea un archivo ejecutable (como los .exe de Windows).
- ◆ **Interpretados:** un programa (intérprete) lee tu código y lo va traduciendo y ejecutando línea a línea, sobre la marcha. Es como tener un traductor simultáneo a tu lado.



Imagen de elaboración propia generada con Claude <<https://claude.ai/>> .
Comparador de lenguajes compilados e interpretados (CC BY-NC-SA
<<http://creativecommons.org/licenses/>>)

Nota aclaratoria: el caso de Java

Aunque en el esquema anterior Java aparece clasificado como lenguaje compilado, conviene matizar este punto. Java sigue en realidad un modelo **híbrido**:

1. El compilador (javac) traduce el código fuente a un código intermedio llamado **bytecode**, no a lenguaje máquina nativo del procesador.
2. Ese bytecode es después ejecutado por la **Máquina Virtual de Java (JVM)**, que lo interpreta y, para mejorar el rendimiento, compila en tiempo de ejecución (JIT, Just-In-Time) las partes del código que se ejecutan con más frecuencia.

Por tanto, Java combina ambos enfoques: hay una fase de compilación (a bytecode) y una fase de interpretación/compilación en tiempo real (por la JVM). Este mismo patrón lo siguen otros lenguajes como C# (con el CLR de .NET) o Kotlin.

La estructura de un programa

Independientemente del lenguaje que uses, cuando pases tu algoritmo a código, la inmensa mayoría de los programas siguen una estructura básica de tres fases:

1. **Entrada de datos (Input):** el programa recibe la información que necesita. Por ejemplo, le pedimos al usuario que introduzca el importe de una factura.
2. **Procesamiento:** el programa aplica la lógica (operaciones, condicionales, bucles) sobre esos datos. Por ejemplo, sumar la factura al total mensual.
3. **Salida de información (Output):** el programa muestra el resultado. Por ejemplo, mostrar en pantalla: "Llevas gastados 150€ este mes".



Imagen de elaboración propia generada con Claude <<https://claude.ai/>> . Estructura de un programa (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

2. Comprueba lo aprendido

Responde a las preguntas del cuestionario

En nuestro reto del Gestor de Gastos, cuando la aplicación muestra en la pantalla de tu móvil una alerta diciendo "¡Cuidado, has superado tu presupuesto!", ¿en qué fase de la estructura del programa nos encontramos?

- ☐ Entrada de datos.
- ☐ Procesamiento.
- ☐ Salida de información.

Si escribimos un programa en un lenguaje de alto nivel y este se va traduciendo y ejecutando línea por línea en el momento, estamos ante un lenguaje...

- ☐ Compilado.
- ☐ Interpretado.

Comprobar

4.2 Tu taller: Python y VSCode

1. Python tu nuevo idioma

Ya sabes que los lenguajes de alto nivel están diseñados para que los entiendas tú, no solo la máquina. Pues bien, entre todos ellos, vas a aprender Python: uno de los más usados del mundo y, no por casualidad, uno de los más sencillos de aprender.

Mira lo sencillo que resulta Python comparado con otros lenguajes. Esta única línea le dice al programa que muestre un mensaje en pantalla. Sin configuraciones, sin complicaciones. Cuando instales tu entorno, será lo primero que escribas.

```
HolaMundo.py X
D: > REA > HolaMundo.py
1 print("Hola, mundo!")
2
3
4
```

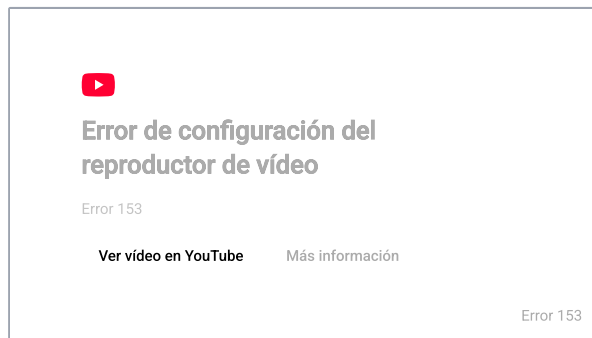
Imagen de elaboración propia. Hola mundo en Python
(CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

Eso es Python: un lenguaje que se lee casi como inglés natural. Fue creado en 1991 por Guido van Rossum con un objetivo muy claro: que programar fuera lo más sencillo y legible posible.

¿Por qué vamos a usar Python?

<https://www.youtube.com/embed/Rv910T1BJUw>
<<https://www.youtube.com/embed/Rv910T1BJUw>>

<<https://www.youtube.com/embed/Rv910T1BJUw>>



Vídeo de Computer Hoy <<https://www.youtube.com/@Computerhoycom>> - ¿Sabes qué es PYTHON y por qué es un lenguaje de programación tan importante?
<<https://www.youtube.com/watch?v=Rv910T1BJUw>> (Licencia de YouTube estándar
<<https://www.youtube.com/static?template=terms>>)

Como has podido ver en el vídeo tenemos varias razones para elegir Python que podríamos resumir en la siguiente imagen:



Imagen de elaboración propia generada con Claude <<https://claude.ai/>> .
Características Python (CC BY-NC-SA <<http://creativecommons.org/licenses/>>)

📌 2. Tu entorno de trabajo

Si quisieras escribir una novela, usarías un procesador de textos como Word o Writer. Para escribir código, necesitas un **Editor de Código**.

Nuestra herramienta principal será **Visual Studio Code** (o simplemente **VSCode**). Piensa en él como un bloc de notas con "superpoderes". Es un programa gratuito, potente y muy utilizado en el mundo profesional.

Para instalarlo, sigue estos pasos:

1. Descarga VS Code desde su página oficial: code.visualstudio.com <<https://code.visualstudio.com/>>
2. Descarga e instala Python desde: <https://www.python.org/downloads/> <<https://www.python.org/downloads/>>
3. Abre VS Code e instala la extensión oficial de Python buscando *Python* en el panel de extensiones.

Si estás trabajando en un ordenador con el sistema operativo EducandOs, busca en el centro de Software (Aplicaciones->Herramientas del Sistema->Centro de Software de EducandOs).

Una vez instalado, este será un entorno de trabajo:



Imagen de creación propia generada con Gemini <<https://gemini.google.com/>> .
Entorno VSCode (CC BY-NC-SA <<http://creativecommons.org/licenses/>>)

📌 Instalación Windows

<https://www.youtube.com/embed/TGb8LVUIUYM>
<<https://www.youtube.com/embed/TGb8LVUIUYM>>

<<https://www.youtube.com/embed/TGb8LVUIUYM>>

Error de configuración del reproductor de vídeo

Error 153

[Ver vídeo en YouTube](#)

[Más información](#)

Error 153

Vídeo de David Toscano. <<https://www.youtube.com/@davidtoscano>> . *Instalar Python en Visual Studio Code* <<https://www.youtube.com/watch?v=TBg8LVUIUYM>> (Licencia de YouTube estándar <<https://www.youtube.com/static?template=terms>>)

Instalación Ubuntu

<https://www.youtube.com/embed/aicZINm0zs4>
<<https://www.youtube.com/embed/aicZINm0zs4>>

<<https://www.youtube.com/embed/aicZINm0zs4>>

Error de configuración del reproductor de vídeo

Error 153

[Ver vídeo en YouTube](#)

[Más información](#)

Error 153

Vídeo de El Rincón del Hacker <<https://www.youtube.com/@elrincondelhacker>> . *Cómo instalar y configurar Python en Linux con Visual Studio Code* <<https://www.youtube.com/watch?v=aicZINm0zs4>> (Licencia de YouTube estándar <<https://www.youtube.com/static?template=terms>>)

3. Comprueba lo aprendido

Relaciona cada tarjeta con su pareja.

5   

Comprobar



Lenguaje de alto nivel

Lenguaje de bajo nivel

Lenguaje compilado

Lee y ejecuta el código línea a línea, sobre la marcha

Diseñado para ser entendido por humanos; usa palabras cercanas al inglés natural

Habla casi directamente el idioma electrónico del ordenador

Python

Lenguaje interpretado

- Traduce el código completo de una vez y genera un archivo ejecutable

- Gratuito, versátil, muy legible y uno de los lenguajes más demandados del mundo

4. Importante

Alto vs. Bajo nivel

Los lenguajes de alto nivel (como el que aprenderemos) están pensados para ser leídos por personas; los de bajo nivel, para ser procesados directamente por el hardware.

Compilado vs. Interpretado

El código compilado se traduce entero de una vez creando un ejecutable. El interpretado se traduce y ejecuta línea a línea.

Estructura E-P-S

Todo programa sigue el ciclo: Entrada (recibir datos) > Proceso (hacer cálculos/lógica) > Salida (mostrar resultados).

5. Los cimientos del código

1. Los datos



Imagen de elaboración propia generada con Gemini <<https://gemini.google.com/>> .
Los cimientos del Gestor de Gastos (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

Ya tienes tu entorno listo y sabes cómo se estructura un programa. Ahora toca aprender el idioma de verdad.

Todo gran proyecto empieza por los cimientos. Cuando un arquitecto diseña una casa, no empieza construyendo el tejado; primero asegura una base sólida que pueda soportar todo el peso de la estructura. En programación, ocurre exactamente lo mismo.

Antes de que nuestro Gestor de Gastos pueda calcular si has ahorrado dinero este mes o avisarte de que te has pasado del presupuesto, necesitas aprender a manejar la materia prima de la informática: **los datos**.

En este bloque vamos a descubrir cuáles son las piezas fundamentales que componen el lenguaje Python. A lo largo de los próximos tres apartados, aprenderás a:

- Crear "cajas" virtuales en la memoria de tu ordenador para guardar información.
- Hacer que el ordenador procese esa información (sumando, restando o comparando gastos).
- Establecer un canal de comunicación directo para que la máquina te pregunte datos y te responda con resultados.

¡Prepara tu entorno de trabajo, porque a partir de aquí empezamos a escribir código de verdad!

2. Objetivos del bloque

Responde todas las preguntas del cuestionario.

En Python, una variable puede cambiar su valor a lo largo del programa, mientras que una constante mantiene siempre el mismo valor.

[Sugerencia](#)

☐ Verdadero ☐ Falso

El operador == en Python asigna un valor a una variable, igual que el operador =.

[Sugerencia](#)

☐ Verdadero ☐ Falso

La función type() en Python permite conocer el tipo de dato que almacena una variable en un momento dado.

[Sugerencia](#)

☐ Verdadero ☐ Falso

Para empezar a programar el Gestor de Gastos, lo primero que haremos será diseñar la pantalla final con los colores y botones que verá el usuario.

[Sugerencia](#)

☐ Verdadero ☐ Falso

5.1 Variables, constantes y tipos de datos

1. Variables y constantes

Para que tu Gestor de Gastos funcione, necesita recordar información: cuánto dinero tienes en la cuenta, qué has comprado o en qué fecha. En programación, para guardar esa información utilizamos **variables**.

Una variable es un espacio en la memoria del ordenador donde puedes guardar un dato y ponerle un nombre para usarlo después. Lo que la hace especial es que, como su nombre indica, su valor puede cambiar a lo largo del programa.

Piensa en una variable como si fuera una caja de mudanza a la que le pones una etiqueta por fuera y le metes algo dentro. En Python, crear una variable es tan fácil como escribir el nombre de la etiqueta, un signo igual (=) y el valor que quieres guardar:

```
saldo_actual = 1500
```

```
concepto = "Factura de la luz"
```

A lo largo del mes, el contenido de esa caja variará (por eso se llaman variables). Sin embargo, a veces necesitamos guardar valores fijos que nunca cambian, como un límite de presupuesto.

A estas "cajas" las llamamos constantes. En Python, para diferenciarlas, las escribimos en mayúsculas: LIMITE_AVISO = 50.

Python técnicamente te permitiría modificar ese valor, pero por convención todos los programadores saben que no deben hacerlo.



Imagen de elaboración propia generada con Gemini <<https://gemini.google.com/>> .
Variable vs. Constante (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

¿Cómo debes nombrar una variable?

No puedes ponerle cualquier nombre a una variable. Python tiene unas reglas que debes respetar:

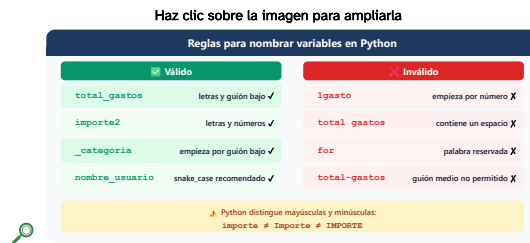


Imagen de elaboración propia generada con Claude <https://claude.ai/> .
 Nomenclatura variables (CC BY-NC-SA <http://creativecommons.org/licenses/>)

Además de las reglas obligatorias, existe una convención muy usada en Python: el estilo snake_case, que consiste en escribir los nombres en minúsculas y separar las palabras con guión bajo:

```
# ✓ Correcto: snake_case
total_gastos = 320.50
nombre_categoria = "ocio"

# ✗ Evitar
TotalGastos = 320.50
nombreCategoria = "ocio"
```

2. Verdadero o falso

Responde todas las preguntas del cuestionario.

En Python, una variable puede comenzar por un número.

[Sugerencia](#)

☐ Verdadero ☐ Falso

Python distingue entre mayúsculas y minúsculas en los nombres de variables.

[Sugerencia](#)

☐ Verdadero ☐ Falso

En Python, las constantes están bloqueadas y el lenguaje impide modificarlas.

[Sugerencia](#)

☐ Verdadero ☐ Falso

La palabra for es un nombre válido para una variable en Python.

[Sugerencia](#)

☐ Verdadero ☐ Falso

Una variable puede cambiar su valor a lo largo de un programa.

[Sugerencia](#)

☐ Verdadero ☐ Falso

3. Tipos de datos

El ordenador necesita saber qué tipo de elemento estás guardando para saber qué puede hacer con él. En nuestro Gestor de Gastos, no tratarás igual el importe de una factura (con el que debes hacer restas) que el nombre de un supermercado. Al igual que no es lo mismo meter agua que arena en una caja. Cada tipo de dato ocupa un espacio diferente en la memoria y permite operaciones distintas.

Python tiene cuatro tipos de datos básicos que usarás constantemente en tu Gestor de Gastos:

- Enteros (int):** números exactos sin decimales. *Ejemplo en el Gestor: el mes del año (mes = 3).*
- Decimales (float):** números con decimales. ¡Ojo! En programación usamos el punto, no la coma. *Ejemplo: el importe de una compra (gasto = 45.50).*
- Cadenas de texto (str):** palabras o frases. Siempre deben ir abrazadas por comillas. *Ejemplo: la categoría (categoria = "Ocio").*
- Booleanos (bool):** solo pueden tener dos estados: Verdadero (True) o Falso (False). *Ejemplo: saber si una factura está pagada (pagado = True).*



Imagen de elaboración propia generada con Claude. Tipos de datos básicos en Python (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

4. ¿Cómo sabe Python qué tipo de dato es cada variable?

A diferencia de otros lenguajes, en Python no necesitas declarar el tipo de dato explícitamente. Python lo detecta de forma automática según el valor que asignas. A esto se le llama **tipado dinámico**:

```
numero_gastos = 5           #Python detecta: int
importe = 45.50             # Python detecta: float
categoria = "comida"        # Python detecta: str
presupuesto_superado = False # Python detecta: bool
```

Si en algún momento quieres comprobar qué tipo de dato tiene una variable, puedes usar la función `type()`:

```
print(type(importe))        # <class 'float'>
print(type(categoria))      # <class 'str'>
```

5. Conversión entre tipos de datos

A veces necesitarás convertir un dato de un tipo a otro. Esto es especialmente importante cuando usas `input()`, ya que todo lo que el usuario escribe llega siempre como texto (str). Si quieres operar con ese valor matemáticamente, deberás convertirlo:

```
# El usuario escribe "45.50" pero Python lo recibe como texto
dato = input("Introduce el importe: ")    # dato = "45.50" (str)
importe = float(dato)                     # importe = 45.50 (float)
```

Recuerda: intentar sumar un número y un texto sin convertir es uno de los errores más frecuentes en Python. El propio intérprete te avisará con un mensaje de error.

6. Completa

Imagina que estás introduciendo el primer movimiento del mes en tu Gestor. Selecciona el tipo de dato correcto para cada variable.

3 0 0 0

Dato 1: gasto = 34.90 La variable gasto es de tipo

Dato 2: tienda = "Mercadona" La variable tienda es de tipo

Dato 3: presupuesto_superado = False La variable presupuesto_superado es de tipo

Comprobar

7. Habla contigo mismo

Cuando escribas código, no solo lo leerá el ordenador: también lo leerás tú mismo, días o semanas después, y quizás otras personas. Los comentarios son líneas que Python ignora completamente al ejecutar el programa, pero que sirven para explicar qué hace cada parte del código.

En Python los comentarios de una sola línea se escriben con `#`:

```
# Inicializamos las variables del gestor de gastos
importe = 45.50          # Importe del gasto en euros
categoria = "comida"      # Categoría a la que pertenece
PRESUPUESTO_MAXIMO = 500 # Constante: presupuesto mensual
```

Para comentarios más largos que ocupen varias líneas se usan tres comillas:

```
'''
Este programa gestiona los gastos personales del usuario.
Permite registrar importes, categorías y fechas,
y avisa cuando se supera el presupuesto mensual.
'''

importe = 45.50
categoria = "comida"
```

Un buen programador no es solo el que escribe código que funciona, sino el que escribe código que cualquiera puede entender. Acostúmbrate a comentar desde el principio.

8. Importante

Variables y constantes

Son los espacios en memoria (cajas) donde guardamos nuestra información. Las variables pueden cambiar de valor durante el programa, las constantes no.

Tipos de datos

Los más comunes son los enteros (*int*), los decimales (*float*), las cadenas de texto (*str*), que se escriben entre comillas, y los booleanos (*bool*), que pueden tomar los valores *True* o *False*."

Comentarios

Se escriben con el símbolo `#`. El ordenador los ignora, pero son vitales para explicar qué hace tu código y que tú (u otra persona) lo entienda en el futuro.

5.2 Operadores y expresiones

1. Operadores

En la página anterior aprendiste a crear "cajas" (variables) para guardar tus datos. Pero, ¿de qué sirve tener el importe de una factura guardado si no puedes hacer nada con él? Para que tu Gestor de Gastos sea útil, necesita procesar esa información: sumar tickets, restar gastos del saldo o comprobar si te has pasado del presupuesto.

Para realizar todas estas acciones, Python utiliza los **operadores**. Piensa en ellos como las herramientas de tu taller: cada uno sirve para una tarea específica. Una **expresión** no es más que una combinación de variables, constantes y operadores que el ordenador evalúa para obtener un resultado final.

Tipos de operadores que usarás en tu reto:

A lo largo de tu proyecto, utilizarás principalmente tres tipos de operadores:

1. Operadores aritméticos (para hacer cuentas).
2. Operadores lógicos (para combinar condiciones).
3. Operadores de comparación (para tomar decisiones).
4. Operadores de asignación.

Son los que ya conoces de las matemáticas de toda la vida. Te permitirán calcular el total gastado o el saldo restante. Aunque la mayoría ya los conoces, en Python alguno tiene una forma diferente a la que estás acostumbrado:



Imagen de elaboración propia generada con
Gemini <<https://gemini.google.com/>> .
Operadores (CC BY-NC-SA
<<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

Haz clic sobre la imagen para ampliarla

Operadores aritméticos en Python			
Op.	Nombre	Ejemplo	Uso en el Gestor
+	Suma	total + importe	Acumular gastos
-	Resta	presupuesto - total	Calcular saldo restante
*	Multiplicación	importe * cantidad	Calcular gasto repetido
/	División	total / num_gastos	Calcular gasto medio
//	División entera	17 // 5 -> 3	Resultado sin decimales
%	Módulo	17 % 5 -> 2	Resto de la división
**	Potencia	2 ** 8 -> 256	Elevar un número a una potencia

Imagen de elaboración propia generada con Claude <<https://claude.ai/>> .
Operadores aritméticos (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

A veces una sola comparación no basta. ¿Y si quieres saber si un gasto es de "Alimentación" y además supera los 100€? Para ello usamos palabras en inglés:

Haz clic sobre la imagen para ampliarla

Operadores lógicos en Python		
and Ambas condiciones deben ser True True and True -> True True and False -> False	or Al menos una condición debe ser True True or False -> True False or False -> False	not Invierte el resultado de la condición not True -> False not False -> True
Ejemplos con el Gestor de Gastos (presupuesto=500, total=480, categoria="ocio", importe=60)		
categoria == "ocio" and importe > 50	-> True and True	-> True
total > 500 or importe > 50	-> False or True	-> True
not (total > presupuesto)	-> not False	-> True

Imagen de elaboración propia generada con Claude <<https://claude.ai/>> .
Operadores lógicos en Python (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

Sirven para comparar dos valores. El resultado siempre será un valor booleano (True o False). Son fundamentales para saber, por ejemplo, si has superado un límite de gasto.

Haz clic sobre la imagen para ampliarla

Operadores de comparación en Python			
Operador	Significado	Ejemplo	Resultado
==	Igual a	total == 480	✓ True
!=	Distinto de	categoria != "ocio"	✗ False
>	Mayor que	total > presupuesto	✗ False
<	Menor que	importe < 100	✓ True
>=	Mayor o igual que	total >= 400	✓ True
<=	Menor o igual que	importe <= 50	✗ False

Imagen de elaboración propia generada con Claude <<https://claude.ai/>> .
Operadores de comparación en Python (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

Ya conoces el operador = para asignar un valor a una variable. Pero Python tiene operadores de asignación compuestos que te permiten actualizar el valor de una variable de forma más compacta. En tu gestor los usarás continuamente para ir acumulando gastos:

Haz clic sobre la imagen para ampliarla

Operadores de asignación en Python		
Asignar o actualizar el valor de una variable		
Op.	Equivalente	Ejemplo con el Gestor
=	Asigna un valor	total = 0
+=	total = total + valor	total += 45.50 -> total = 45.50
-=	total = total - valor	presupuesto -= 45.50
*=	total = total * valor	importe *= 2
/=	total = total / valor	media /= num_gastos
Acumulando gastos paso a paso con +=		
Paso 1	Paso 2: total += 45.50	Paso 3: total += 32.00
total = 0	total = 45.50	total = 77.50
Paso 4: total += 18.75		
total = 96.25 € ✓		

Imagen de elaboración propia generada con Claude <<https://claude.ai/>> .
Operadores de asignación (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

2. ¡Ojo con estos operadores!

Fíjate especialmente en estos tres, que suelen sorprender a los principiantes:

```
# División entera: devuelve solo la parte entera
print(17 // 5)      # Resultado: 3

# Módulo: devuelve el resto de la división
print(17 % 5)       # Resultado: 2

# Potencia
print(2 ** 8)        # Resultado: 256
```

En el Gestor de Gastos, el operador % te será muy útil para saber, por ejemplo, si el número de gastos es par o impar, y el // para calcular medias enteras.



3. Comprueba lo aprendido

Elija la respuesta correcta

10

00:00



[Pulse aquí para jugar](#)



4. Hablando con el usuario

Ya sabes guardar datos en variables, operar con ellos y tomar decisiones lógicas. Pero un programa que no se comunica con el usuario no sirve de mucho. Necesitamos que tu Gestor de Gastos hable contigo: que te pregunte cuánto ha costado el café de esta mañana y te informe rápidamente de cuánto dinero te queda.

En Python eso se hace con `input()` y `print()`.

Sirve para mostrar mensajes o el valor de tus variables en la pantalla. Imagina que es la "boca" de tu programa. Para usarla, solo tienes que poner entre paréntesis lo que quieres decir. Si es un texto fijo, va entre comillas; si es una variable, va sin comillas.

```
saldo_actual=1200
total=500
print("¡Atención! Te estás acercando al límite")
print(saldo_actual)
print(f"Total gastado este mes: {total} €") # Con f string combina texto y variables en Python
```

Esta es la "oreja" de tu programa. Sirve para que el ordenador pause la ejecución y se quede esperando a que tú (el usuario) escribas algo en el teclado y pulses la tecla Enter. Lo que el usuario escriba, normalmente lo guardaremos dentro de una variable.

```
#Pedir el importe y convertirlo a número decimal
importe = float(input("Introduce el importe del gasto: "))

#Pedir la categoría (texto, no necesita conversión)
categoria = input("Introduce la categoría (comida, ocio...): ")

#Pedir el presupuesto mensual y convertirlo a entero
presupuesto = int(input("¿Cuál es tu presupuesto mensual? "))
```

Recuerda: `input()` siempre devuelve un str. Si intentas sumar ese valor directamente sin convertirlo obtendrás un error.



5. Ordena el programa

El siguiente programa pide al usuario un gasto, lo suma al total y muestra el resultado. Ordena las líneas correctamente, según el orden que se da en estas instrucciones:

```
print(f"Total acumulado: {total} €")
```

```
importe = float(input("Introduce el importe del gasto: "))
```

```
total = 0
```

```
total += importe
```

Comprobar



6. ¡Hola Gestor!

¡Ha llegado el momento de mancharse las manos de código! Vamos a juntar todo lo que has aprendido en este bloque para crear un "mini-gestor" de un solo uso.

Abre tu entorno de trabajo (VSCode) y crea un archivo nuevo llamado `primer_gasto.py`. Tu objetivo es escribir un programa que haga exactamente lo siguiente en este orden:

1. **Salude** al usuario (usando `print`).
2. **Pregunte** de cuánto presupuesto dispone y lo guarde en una variable. (¡Ojo! Recuerda convertir la respuesta a número decimal con `float`).
3. **Pregunte** cuánto le ha costado el desayuno de hoy y lo guarde en otra variable.
4. **Calcule** matemáticamente cuánto dinero le queda y lo guarde en una tercera variable llamada `saldo_restante`.
5. **Muestre** un mensaje final en pantalla diciendo el dinero que le sobra.

¡Inténtalo tú solo! No pasa nada si sale un error en la pantalla negra (el Terminal). Los errores son los mejores maestros del programador; léelos, busca dónde te falta un paréntesis o unas comillas, y vuelve a ejecutarlo.

¿Te has atascado o quieres comprobar tu solución? Haz clic en el botón de abajo para ver cómo lo haría un programador profesional. Recuerda que en programación hay muchas formas de llegar al mismo resultado, así que tu código no tiene que ser idénticamente exacto letra por letra, pero sí seguir esta lógica.

```
# 1. Saludamos
print("¡Hola! Bienvenido a tu calculadora rápida.")

# 2. Pedimos los datos numéricos (usando float para los decimales)
presupuesto = float(input("¿De cuánto presupuesto dispones hoy? "))
gasto = float(input("¿Cuánto te ha costado el desayuno? "))

# 3. Hacemos el cálculo
saldo_restante = presupuesto - gasto

# 4. Mostramos el resultado.
# (Nota: podemos separar texto y variables usando una coma)
print("Tras el desayuno, te quedan:", saldo_restante, "euros.")
```



7. Importante



Operadores aritméticos

Son las herramientas matemáticas (+, -, *, /) que te permiten realizar los cálculos numéricos de tu Gestor de Gastos (sumar totales, restar del saldo).



Operadores de comparación y lógicos

Los de comparación (==, !=, >, etc.) comparan valores devolviendo True o False. Los lógicos (and, or, not) permiten combinar estas comparaciones para crear condiciones más complejas (ej: gasto alto Y categoría ocio).

Mostrar datos (print)

La orden print() muestra información en la pantalla. Puedes usarla para texto fijo o para ver qué hay dentro de tus variables.

Pedir datos (input)

La orden input() permite al usuario introducir información a través del teclado, guardando la respuesta en una variable.

Conversión de tipos

Recuerda que input() siempre captura texto. Si vas a pedir un número para hacer cálculos en tu Gestor, debes convertirlo a número entero o decimal envolviéndolo con int() o float(). Ejemplo: float(input("Importe: "))

6. El programa decide y repite

1. Estructuras de control

Hasta este momento, los pequeños programas que has imaginado o escrito funcionan como una cascada de agua: empiezan en la primera línea de código y caen hacia abajo, ejecutando una instrucción tras otra hasta llegar al final. A esto se le llama **programación secuencial** y, aunque es el punto de partida, tiene una limitación enorme: **la vida real no es lineal**.

A veces tienes que tomar decisiones basadas en lo que ocurre: *"Si llueve, entonces cojo el paraguas; si no, me pongo gafas de sol"*. Otras veces, tienes que repetir la misma acción muchas veces: *"Mientras queden platos sucios en el fregadero, sigo lavando"*.

Piensa en tu Gestor de Gastos. Necesitas que haga cosas distintas según la situación:

- Si el gasto supera el presupuesto, mostrar una alerta. Si no, confirmarlo.
- Repetir el proceso de registro para cada gasto del mes sin escribir el mismo código veinte veces.
- Seguir mostrando el menú hasta que el usuario decida salir.

Nada de esto es posible solo con secuencias. Necesitas que tu programa sea capaz de **tomar decisiones** y de **repetir acciones** de forma automática. Para eso existen las **estructuras de control**:

1. **Estructuras condicionales (las decisiones)**: permiten al programa elegir entre varios caminos diferentes.
2. **Estructuras iterativas o bucles (las repeticiones)**: permiten al programa repetir un bloque de código las veces que haga falta.

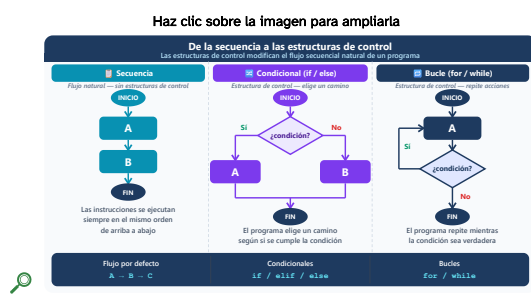


Imagen de elaboración propia generada con Claude. Estructuras de control (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

Con ellas, tu código deja de ser una lista rígida de pasos y se convierte en un programa que reacciona, que se adapta y que trabaja por ti. En este bloque aprenderás a usarlas todas.

2. ¿Preparados?

Vamos a comprobar si has pillado la idea principal antes de meternos a escribir código. Imagina que quieres que tu Gestor de Gastos te pida la contraseña al abrir la aplicación. Si la pones mal, te la vuelve a pedir. ¿Qué tipo de estructura necesitará el programa para hacer esto?

- ☐ Una estructura puramente secuencial.
- ☐ Una estructura de repetición (bucle).
- ☐ No hace falta ninguna estructura, Python lo hace solo.

Comprobar

6.1 Toma de decisiones

📌 1. Estructuras condicionales

En tu día a día, tomas cientos de decisiones basadas en condiciones: "Si tengo saldo en la cuenta, compro las entradas; si no, espero al mes que viene". En Python, le enseñamos al ordenador a pensar exactamente de la misma manera utilizando la estructura `if` (que en inglés significa "sí...", de condición).



Imagen de elaboración propia generada con Gemini
<<https://gemini.google.com/>> . Estructuras de control (CC BY-NC-SA
<<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

Vamos a ver las tres piezas clave para construir la lógica de tu Gestor de Gastos:

Es la regla más básica. Le decimos al programa que compruebe una expresión matemática o lógica. Solo si el resultado es verdadero (True), ejecutará las instrucciones que vienen justo debajo.

Imagina que tienes un presupuesto de 100€ y acabas de introducir un ticket de 120€. El programa debería darte un aviso de alerta:

```
presupuesto = 100
gasto = 120

if gasto > presupuesto:
    print("¡Alerta roja! Has superado tu presupuesto mensual.")
```

¿Qué pasa si la condición del `if` no se cumple (por ejemplo, el gasto es de solo 40€)? El programa, por defecto, se salta ese bloque y no hace nada. Pero a veces queremos que haga otra cosa en ese caso. Para eso usamos la palabra `else`.

```
presupuesto = 100
gasto = 120

if gasto > presupuesto:
    print("¡Alerta roja! Has superado tu presupuesto mensual.")
else:
    print("¡Todo en orden! Sigues dentro del límite.")
```

El `if` y el `else` te permiten elegir entre dos caminos. Pero, ¿qué pasa cuando hay más de dos posibilidades? Imagina que quieres clasificar tus gastos en tres niveles según el importe: bajo, moderado o elevado. Para eso existe el `elif`.

```
importe = 75

if importe < 20:
    print("♥ Gasto bajo")
elif importe < 50:
    print("🟡 Gasto moderado")
elif importe < 100:
    print("🔴 Gasto elevado")
else:
    print("🔴 Gasto muy elevado: ¿es necesario?")
```

Python evalúa las condiciones en orden, de arriba a abajo. En cuanto encuentra una que se cumple, ejecuta su bloque y salta el resto. Por eso el orden importa: coloca siempre las condiciones de más restrictiva a menos restrictiva.



Imagen de elaboración propia generada con Claude <<https://claude.ai/>> .
Comparativa if, else, elif (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

A veces una sola condición no es suficiente para tomar una decisión. En tu Gestor de Gastos puede que necesites comprobar varias cosas a la vez: que el gasto sea elevado **y** que además pertenezca a la categoría de ocio, **o** que se haya superado el presupuesto **o** que el número de gastos sea muy alto. Para eso puedes encadenar condiciones usando los operadores lógicos **and** y **or** que ya conoces:

```
importe = 75
categoria = "ocio"
total = 480
presupuesto = 500

# AND: se deben cumplir AMBAS condiciones
if categoria == "ocio" and importe > 50:
    print("Gasto de ocio elevado: revisa si es necesario")

# OR: basta con que se cumpla UNA de las dos
if total > 450 or importe > 100:
    print("Atención: estás cerca del límite mensual")

# Combinación de AND y OR con paréntesis
if (categoria == "ocio" and importe > 50) or total > presupuesto:
    print("Alerta de gasto: revisa tu presupuesto")
```

Cuando combinas **and** y **or** en una misma condición, usa paréntesis para dejar claro el orden de evaluación. Igual que en matemáticas, los paréntesis se evalúan primero y evitan resultados inesperados.

Recuerda: con **and** ambas condiciones deben ser **True** para que el bloque se ejecute. Con **or** basta con que una lo sea.



Seguramente te habrás dado cuenta de que en el código también podemos añadir emojis que hará que la salida por el terminal sea más atractiva.

Si quieres añadirlos en tu código puedes entrar en [Emojipedia <https://emojipedia.org/>](https://emojipedia.org/), busca el emoji que quieras y cópialo. Para añadirlos a tu código simplemente los pegas dentro de la cadena de texto en VS Code y funcionan perfectamente.

Para que los emojis se muestren correctamente en la terminal de VS Code asegúrate de que el archivo está guardado con codificación UTF-8. En VS Code se ve en la barra inferior derecha. Si pone otra cosa haz clic y cámbialo a UTF-8.

📌 2. La regla de oro de Python: la indentación

Fíjate muy bien en los ejemplos de código anteriores. Las órdenes `print()` que van por dentro del `if`, `elif` o `else` no empiezan pegadas al margen izquierdo, sino que tienen un espacio hacia la derecha (una tabulación o sangría). Así es como Python sabe visualmente qué instrucciones pertenecen a la condición. Si olvidas esta sangría, VSCode te marcará la línea en rojo y el programa dará error. Esta tabulación se consigue dejando 4 espacios al comienzo de cada línea o pulsando la tecla tabulador.

```
presupuesto = 500
total = 320

# ✅ Correcto
```

```

if total > presupuesto:
    print("Presupuesto superado") # 4 espacios de sangría
    print("Revisa tus gastos")    # mismo nivel

# ❌ Error: falta indentación
if total > presupuesto:
    print("Presupuesto superado") # Python no sabe que pertenece al if

```

VS Code añade la indentación automáticamente al pulsar Enter después de los dos puntos. Si ves que algo no funciona, revisa siempre la sangría primero.



3. Para saber más

- 📌 Artículo de **freeCodeCamp** <<https://www.freecodecamp.org/espanol/news/sentencia-if-else-de-python-explicacion-de-las-sentencias-condiciones/>> : Una explicación paso a paso y muy visual sobre las sentencias *if/else*, ideal para principiantes.
- 📌 ¡Practica! Ejercicios resueltos de estructuras de control en Python. <<https://codigodev.com/ejercicios-resueltos-de-estructuras-de-control-en-python/>>



4. Revisa lo aprendido

Responde todas las preguntas del cuestionario.

El bloque else es obligatorio siempre que uses un if.

[Sugerencia](#)

☐ Verdadero ☐ Falso

En Python, la indentación es solo una cuestión de estilo visual.

[Sugerencia](#)

☐ Verdadero ☐ Falso

La condición de un if debe devolver siempre True o False.

[Sugerencia](#)

☐ Verdadero ☐ Falso

El siguiente código es correcto en Python:

```

if total > 500:
    print("Alerta")

```

[Sugerencia](#)

☐ Verdadero ☐ Falso



5. El guardián de tu cartera

¿Recuerdas el programa ¡Hola, Gestor! que escribiste antes? Ha llegado el momento de mejorarlo. Vas a programar el sistema de avisos de tu Gestor de Gastos siguiendo estos pasos:

1. Crea un archivo llamado *alertas.py*. Tu objetivo es programar el sistema de avisos de tu Gestor de Gastos.
2. Crea una variable llamada *saldo* y asigne el valor de tu dinero actual (por ejemplo, 50).
3. Pide al usuario por teclado (usando *input* y *float*) que introduzca el importe de su última compra y guárdalo en la variable *compra*.
4. Crea la estructura de decisión:
 - **SI** la compra es mayor que el saldo, muestra en pantalla: "Operación denegada. No tienes fondos suficientes."
 - **SI NO**, calcula el nuevo saldo (restando la compra al saldo), y muestra en pantalla: "Compra aceptada. Tu nuevo saldo es de X euros."

¡Inténtalo sin mirar la solución! Ejecuta tu código un par de veces introduciendo números diferentes para comprobar que el programa sabe tomar ambos caminos de forma inteligente.

```
# 1. Establecemos nuestro saldo inicial
saldo = 50.0

# 2. Pedimos el gasto al usuario (convertido a decimal)
compra = float(input("¿De cuánto es tu ticket de compra?: "))

# 3. Estructura de decisión
if compra > saldo:
    print("Operación denegada. No tienes fondos suficientes.")
else:
    saldo = saldo - compra
    print("Compra aceptada. Tu nuevo saldo es de:", saldo, "euros.")
```



6. Importante

El bloque if

Es la estructura principal. Evalúa una condición y, si es verdadera (true), ejecuta las instrucciones tabuladas en su interior.

Las alternativas elif y else

Se utilizan justo después de un if. Elif permite añadir más comprobaciones encadenadas, y else se ejecuta automáticamente como último recurso cuando ninguna de las condiciones anteriores se ha cumplido.

La indentación

En Python es obligatorio utilizar una sangría (indentación) en el código que va dentro de las estructuras de control. Esta sangría se consigue normalmente dejando cuatro espacios al comienzo de cada línea y sirve para indicar qué instrucciones pertenecen a cada bloque de código. Otros lenguajes de programación utilizan llaves {} para delimitar estos bloques.

Condiciones encadenadas (and / or)

A veces necesitas que tu programa compruebe varias cosas al mismo tiempo antes de tomar una decisión. Para unir varias condiciones en una sola línea, utilizamos los operadores lógicos:

- ◆ **and (Y lógico):** obliga a que todas las condiciones sean verdaderas para que se ejecute el código. Es ideal para ser muy específico.
 - Ejemplo: avisar solo si el gasto es alto Y además es de una categoría prescindible.
 - if compra > 100 and categoría == "Ocio":
- ◆ **or (O lógico):** basta con que al menos una de las condiciones sea verdadera. Es muy útil para agrupar opciones.
 - Ejemplo: clasificar un ticket como "Gasto del Hogar" si es de luz o si es de agua.
 - if recibo == "Luz" or recibo == "Agua":

6.2 Repite sin cansarte



1. Estructuras iterativas

Imagina que llegas a casa con 10 tickets de compra en el bolsillo y quieres registrarlos en tu Gestor de Gastos. Con lo que sabemos hasta ahora, tendrías que escribir la instrucción input() diez veces seguidas en tu código, una debajo de otra. Y si un día traes 15 tickets, ¡el programa se quedaría corto!

Para solucionar esto de forma elegante, la programación estructurada nos ofrece los bucles o estructuras iterativas. Son bloques de código que se repiten a sí mismos de forma automática. En Python, utilizamos dos tipos principales dependiendo de lo que necesitemos:

Imagina que quieres mostrar el resumen de todos los gastos que has registrado en el mes. Sin bucles, tendrías que escribir un `print()` por cada gasto. Si tienes 20 gastos, 20 líneas de código. Si tienes 100, 100 líneas. Claramente eso no es programar bien.

Los **bucles** son la solución: permiten repetir un bloque de código automáticamente un número determinado de veces o para cada elemento de una colección. El bucle `for` es el más usado cuando **sabes de antemano cuántas veces quieres repetir** o cuando quieres recorrer una lista de elementos.

```
# Estructura básica

for variable in secuencia:
    # bloque que se repite
```

En el siguiente ejemplo podemos ver que la variable `i` toma automáticamente cada valor de la secuencia en cada repetición. Puedes llamarla como quieras, aunque `i` es la convención más habitual.

```
# Ejemplo: mostrar 5 mensajes

for i in range(5):
    print(f"Gasto número {i+1} registrado")
```

La función `range()`

`range()` genera una secuencia de números que el `for` recorre uno a uno. Es una de las herramientas más usadas junto con los bucles:

Haz clic sobre la imagen para ampliarla

La función range() en Python Genera secuencias de números para usar con el bucle for			
Uso	Sintaxis	Secuencia generada	Ejemplo con for
Desde 0 hasta n-1 (un solo argumento)	<code>range(5)</code>	0, 1, 2, 3, 4	<code>for i in range(5): print(i)</code>
Desde inicio hasta fin-1	<code>range(1, 6)</code>	1, 2, 3, 4, 5	<code>for i in range(1, 6): print(i)</code>
Desde inicio hasta fin-1, de X en X	<code>range(0, 10, 2)</code>	0, 2, 4, 6, 8	<code>for i in range(0, 10, 2): print(i)</code>
Cuenta hacia atrás (paso negativo)	<code>range(5, 0, -1)</code>	5, 4, 3, 2, 1	<code>for i in range(5, 0, -1): print(i)</code>

Aplicado al Gestor de Gastos

```
for i in range(1, 6):
    ... Pide 5 gastos al usuario numerados del 1 al 5
```

Imagen de elaboración propia generada con Claude <<https://claude.ai/>>. Función `range()` (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

El bucle `for` es perfecto cuando sabes cuántas veces quieres repetir algo. Pero, ¿qué pasa cuando no lo sabes? Imagina el menú de tu Gestor de Gastos: no puedes saber cuántas veces el usuario querrá registrar un gasto antes de salir. Para eso existe el `while`.

La palabra `while` significa "mientras". El programa comprobará una condición y, si es verdadera (`True`), ejecutará el bloque de código indentado. Al terminar, volverá a subir para comprobar la condición otra vez. Y así sucesivamente hasta que la condición pase a ser falsa (`False`).

```
# Estructura básica

while condicion:
    # bloque que se repite
```

Es perfecto para crear el Menú Principal de tu Gestor de Gastos. Queremos que el menú siga apareciendo mientras el usuario no pulse la opción de salir.

```
# El menú se repite hasta que el usuario elija salir
opcion = ""

while opcion != "3":
    print("\n--- MENÚ DEL GESTOR ---")
    print("1. Registrar gasto")
    print("2. Ver total")
    print("3. Salir")
    opcion = input("Elige una opción: ")
```

Si la condición del `while` nunca se vuelve `False`, el programa se queda atrapado en un bucle infinito y nunca termina. Asegúrate siempre de que algo dentro del bucle cambia el valor de la condición.

Contadores y acumuladores: los mejores amigos del bucle

Una de las aplicaciones más prácticas que le darás a la estructura `while` es utilizarla para llevar el recuento de lo que ocurre dentro de tu programa. Para ello, los programadores utilizamos dos variables con misiones muy específicas:

- 📌 **El contador**, como su propio nombre indica, es una variable que va incrementando su valor de 1 en 1. Su objetivo es averiguar cuántas veces ha ocurrido algo (por ejemplo, saber cuántos tickets de compra has registrado hoy).

- 📌 **El acumulador:** a diferencia del contador, esta variable no suma de 1 en 1, sino que almacena y va sumando cantidades diferentes para guardar un "total" progresivo (por ejemplo, ir sumando el dinero de cada ticket para calcular el gasto total del día).

¿Quieres ver cómo trabajan juntos? Fíjate en este pequeño programa para tu Gestor que utiliza un bucle while junto a un contador y un acumulador:

```
# Inicializamos las variables a cero antes de empezar
contador_tickets = 0
total_acumulado = 0.0
opcion = "Sí"

# El bucle se repite mientras el usuario diga "Sí"
while opcion == "Sí":
    importe = float(input("Introduce el importe del ticket: "))

    # ACUMULADOR: Al total que ya teníamos, le sumamos el nuevo importe
    total_acumulado = total_acumulado + importe

    # CONTADOR: Sumamos 1 a la cantidad de tickets registrados
    contador_tickets = contador_tickets + 1

    opcion = input("¿Quieres introducir otro ticket? (Sí/No): ")

# Cuando el usuario escribe "No", el bucle termina y mostramos el resultado final:
print("Hoy has registrado", contador_tickets, "tickets.")
print("El gasto total ha sido de", total_acumulado, "euros.")
```

Si creas un bucle *while* pero olvidas incluir una instrucción en su interior que haga que la condición cambie en algún momento, el programa se quedará atascado repitiendo eso para siempre (o hasta que el ordenador se bloquee). ¡Asegúrate siempre de que el usuario tenga una vía de escape!

A veces, en programación se usan de forma controlada y deliberada, por ejemplo para mantener un menú activo hasta que el usuario decida salir:

```
while True:
    print("El programa sigue en marcha...")
```

Otro caso habitual de bucle infinito ocurre cuando la variable que controla la condición **nunca se actualiza** dentro del bucle. Es uno de los errores más frecuentes entre los principiantes:

```
# ❌ Bucle infinito por error: el contador nunca cambia
contador = 0

while contador < 5:
    importe = float(input(f"Introduce el gasto {contador + 1}: "))
    print(f"Gasto de {importe} € registrado")
    # ¡Falta contador += 1!
    # Como contador siempre vale 0, la condición
    # contador < 5 siempre es True y el bucle nunca termina
```

Para corregir el bucle anterior, tendríamos que actualizar la variable contador dentro del bucle:

```
# ✅ Correcto: el contador se actualiza en cada iteración
contador = 0

while contador < 5:
    importe = float(input(f"Introduce el gasto {contador + 1}: "))
    print(f"Gasto de {importe} € registrado")
    contador += 1 # Ahora sí avanza hacia el final del bucle
```

Si tu programa se queda "colgado" al ejecutario, lo primero que debes revisar es si la condición del while puede volverse False en algún momento. Si no, has creado un bucle infinito sin querer. En VS Code puedes detenerlo pulsando Ctrl + C en el terminal.

Controlando el flujo del bucle: *break* y *continue*

Sin ningún mecanismo de salida, el primer programa nunca terminaría. Para detenerlo tendrías que cerrarlo a la fuerza (En VS Code puedes detenerlo pulsando Ctrl + C en el terminal). Por eso los bucles infinitos siempre van acompañados de un *break*.

- 📌 **break** interrumpe el bucle inmediatamente y sale de él, aunque la condición siga siendo verdadera. Es la puerta de salida de un bucle infinito:

```
# Parar de registrar gastos si el importe es 0
while True:
    importe = float(input("Introduce un gasto (0 para salir): "))
    if importe == 0:
        print("Saliendo del gestor...")
        break
    print(f"Gasto de {importe} € registrado")
```

- 📌 **continue** no sale del bucle, sino que salta el resto de la iteración actual y pasa directamente a la siguiente. Es útil para ignorar casos que no quieres procesar:

```
gastos = [45.50, -10.00, 32.00, -5.00, 18.75]

# Ignorar los gastos negativos (errores de entrada)
for gasto in gastos:
    if gasto < 0:
        print(f"Gasto inválido ({gasto} €), ignorado")
        continue
    print(f"Gasto registrado: {gasto} €")
```

Recuerda: *break* sale del bucle, *continue* salta a la siguiente iteración. Son herramientas distintas para situaciones distintas.

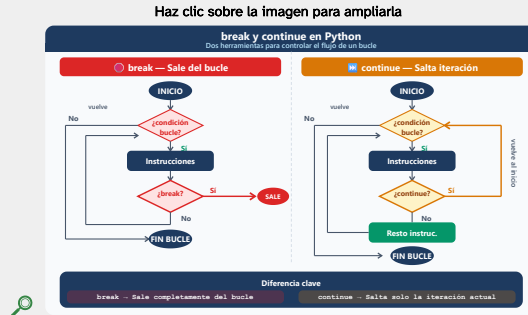


Imagen de elaboración propia generada con Claude <<https://claude.ai/>> . Break vs. Continue (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

2. Relaciona

Relaciona el bucle con su uso

3

Comprobar

Ejecutar una acción un número concreto de veces

Pedir 5 gastos al usuario

Repetir mientras el total no supere el presupuesto

for con range()

while

for con range(5)

3. Pasando lista con for

Donde el *for* brilla especialmente es recorriendo listas de elementos. En tu Gestor de Gastos lo usarás para procesar todos los gastos registrados:

```
gastos = [45.50, 32.00, 18.75, 120.00, 9.99]

# Mostrar todos los gastos
for gasto in gastos:
    print(f"Gasto: {gasto} €")

# Sumar todos los gastos
total = 0
for gasto in gastos:
    total += gasto
print(f"Total del mes: {total} €")
```

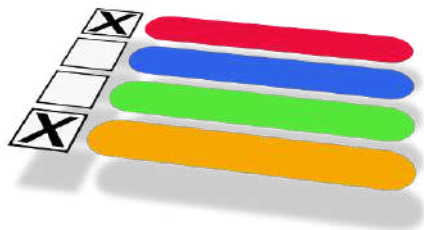


4. Elige la respuesta correcta

Lee los siguientes escenarios que podrían ocurrir en tu Gestor de Gastos e indica si sería más eficiente programarlos usando un bucle *while* o un bucle *for*

3 0 0

00:00



[Pulse aquí para jugar](#)



5. Suma todos los gastos

Escribe un programa que pida al usuario 5 importes de gastos uno a uno usando un bucle *for*, los vaya acumulando en una variable *total* y al final muestre el total gastado y la media por gasto.

La salida de tu programa debería ser similar a esta:

```
Introduce el gasto 1: 45.50
Introduce el gasto 2: 32.00
Introduce el gasto 3: 18.75
Introduce el gasto 4: 120.00
Introduce el gasto 5: 9.99
```

```
Total gastado: 226.24 €
Media por gasto: 45.25 €
```

Imagen de elaboración propia. Salida Terminal (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/>>)

```
# Suma de gastos con bucle for
total = 0

for i in range(5):
    gasto = float(input(f"Introduce el gasto {i + 1}: "))
    total += gasto

media = total / 5

print()
print(f"Total gastado: {total:.2f} €")
print(f"Media por gasto: {media:.2f} €")
```



Reto extra

Modifica el programa para que en lugar de pedir siempre 5 gastos, pregunte primero al usuario cuántos gastos quiere introducir y use ese número en el *range()*.



6. Menú interactivo del gestor

Es el momento de dar un gran salto en tu Gestor de Gastos. Escribe un programa que muestre un menú interactivo con tres opciones y que se repita hasta que el usuario elija salir:

- ◆ **Opción 1** — Registrar un gasto: pide el importe y la categoría y los muestra por pantalla.
- ◆ **Opción 2** — Ver el total acumulado: muestra la suma de todos los gastos registrados.
- ◆ **Opción 3** — Salir: muestra un mensaje de despedida y termina el programa.

La salida de tu programa debería ser similar a esta:

```
--- MENÚ DEL GESTOR ---
1. Registrar gasto
2. Ver total acumulado
3. Salir
Elige una opción: 1
Importe del gasto: 45.50
Categoría: comida
Gasto de 45.5 € en comida registrado.

--- MENÚ DEL GESTOR ---
1. Registrar gasto
2. Ver total acumulado
3. Salir
Elige una opción: 2
Total acumulado: 45,5 €

--- MENÚ DEL GESTOR ---
1. Registrar gasto
2. Ver total acumulado
3. Salir
Elige una opción: 3
¡Hasta pronto! Tu gestor ha terminado.
```

Imagen de elaboración propia. Salida Terminal (CC BY-NC-SA
<<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)



Usa un `while True` con un `break` para salir cuando el usuario elija la opción 3. Usa una variable total que vaya acumulando los gastos con `+=`.

```
# Menú interactivo del Gestor de Gastos

total = 0

while True:
    print("\n--- MENÚ DEL GESTOR ---")
    print("1. Registrar gasto")
    print("2. Ver total acumulado")
    print("3. Salir")
    opcion = input("Elige una opción: ")

    if opcion == "1":
        importe = float(input("Importe del gasto: "))
        categoria = input("Categoría: ")
        total += importe
        print(f"Gasto de {importe} € en {categoria} registrado.")

    elif opcion == "2":
        print(f"Total acumulado: {total} €")

    elif opcion == "3":
        break
```

```
elif opcion == "3":
    print(" ¡Hasta pronto! Tu gestor ha terminado.")
    break
else:
    print(" Opción no válida. Elige 1, 2 o 3.")
```

📖 Reto extra

Reto extra: Añade una cuarta opción al menú que muestre el gasto más alto registrado hasta el momento. Para ello necesitarás guardar cada importe en una lista y usar la función `max()`.

📍 7. Importante

📖 El bucle while (Mientras)

Se utiliza cuando no sabemos cuántas veces se va a repetir una acción. El bloque de código se repite indefinidamente mientras una condición previa siga siendo verdadera. Es fundamental asegurar que la condición se vuelva falsa en algún punto para evitar bucles infinitos.

📖 El bucle for (Para cada)

Se utiliza cuando conocemos de antemano el número exacto de iteraciones. Recorre secuencias de elementos (como listas, que veremos más adelante) o rangos numéricos utilizando `range()`

📖 La indentación en los bucles

Al igual que ocurría con los condicionales (if), todo el código que quieras que se repita dentro de un bucle while o for debe estar obligatoriamente tabulado hacia la derecha.

7. Organiza tus datos

📖 1. Introducción



Imagen de elaboración propia generada con Gemini <<https://gemini.google.com/>> .
Variable vs. Listas (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

A lo largo de los apartados anteriores hemos utilizado las variables como "cajas de mudanza" para guardar nuestros datos. Sin embargo, estas cajas tienen un pequeño problema: solo pueden guardar una cosa a la vez. En la vida real los programas manejan colecciones de datos: todos los gastos del mes, las categorías disponibles, el resumen por tipo de gasto...

Imagina que quieres registrar 30 gastos en tu Gestor. ¿Crearías 30 variables distintas? `gasto1`, `gasto2`, `gasto3`... Claramente no. Para eso existen las **estructuras de datos**: contenedores que te permiten agrupar múltiples valores bajo un mismo nombre y trabajar con ellos de forma organizada y eficiente.

Python ofrece cuatro estructuras de datos fundamentales, cada una diseñada para una situación distinta.





Elegir la estructura adecuada para cada situación es una de las decisiones más importantes que tomarás como programador. A lo largo de este bloque conocerás cada una de ellas y aprenderás cuándo usar cada una en tu Gestor de Gastos.

📌 2. Datos en equipo: listas y tuplas

Una lista es una colección ordenada de elementos que puedes modificar en cualquier momento: añadir nuevos datos, eliminar los que ya no necesitas o cambiar cualquier valor. En tu Gestor de Gastos, una lista es perfecta para almacenar todos los importes del mes:

```
# Crear una lista de gastos
gastos = [45.50, 32.00, 18.75, 120.00, 9.99]

# Acceder a un elemento por su posición (índice desde 0)
print(gastos[0]) # 45.50 – el primero
print(gastos[-1]) # 9.99 – el último

# Añadir un nuevo gasto
gastos.append(67.30)

# Eliminar un gasto
gastos.remove(32.00)

# Longitud de la lista
print(len(gastos)) # Número de gastos registrados

# Recorrer todos los gastos
for gasto in gastos:
    print(f"{gasto} €")
```

¿Y una tupla?

Una tupla es igual que una lista pero inmutable: una vez creada no puedes modificarla. Úsala cuando los datos no deben cambiar, como las categorías disponibles del sistema:

```
# Tupla de categorías fijas del gestor
CATEGORIAS = ("comida", "transporte", "ocio", "casa", "salud")

print(CATEGORIAS[0]) # comida
print(len(CATEGORIAS)) # 5

# Esto daría error: las tuplas no se pueden modificar
# CATEGORIAS.append("viajes") ❌
```



¿Lista o tupla? Usa una lista cuando los datos van a cambiar a lo largo del programa. Usa una tupla cuando los datos son fijos y no deben modificarse. Las tuplas además son más rápidas y consumen menos memoria.

Haz clic sobre la imagen para ampliar

Lista vs Tupla en Python		
Misma estructura, comportamiento distinto		
Operación	[] Lista	() Tupla
Crear	gastos = [45.50, 32.00, 18.75] gastos = list()	categorias = ("comida", "ocio") categorias = tuple()
Acceder	gastos[0] → 45.50 gastos[-1] → 18.75 (último)	categorias[0] → "comida" categorias[-1] → "ocio"
Modificar	✓ Permitido gastos[0] = 50.00	X No permitido — son inmutables categorias = ("comida", "ocio", "casa")
Añadir / Eliminar	gastos.append(67.30) gastos.remove(32.00)	X No se puede modificar
Recorrer	for gasto in gastos: print(gasto)	for cat in categorias: print(cat)
Convertir	tuple(gastos) → convierte a tupla	list(categorias) → convierte a lista

🔍 Usa lista cuando los datos cambian - Usa tupla cuando los datos son fijos

Imagen de elaboración propia generada con Claude <https://claude.ai/> . Listas vs. Tuplas (CC BY-NC-SA <http://creativecommons.org/licenses/>)

3. Para saber más

Puedes consultar estos dos recursos oficiales para conocer más sobre las listas y tuplas.

- ◆ Listas en Python — w3schools.com <https://www.w3schools.com/python/python_lists.asp>
- ◆ Listas y tuplas — docs.python.org <https://docs.python.org/es/3/tutorial/datastructures.html#more-on-lists>

4. Practicamos lo aprendido

Elija la respuesta correcta

10 🟢 0 🔴 0 🏆 0

🕒 00:00 🔄 🔄



Pulsa aquí para jugar

5. Más estructuras: dict y set

Las listas están genial para guardar un historial de números, pero a veces la información es un poco más compleja. Imagina que quieres guardar un gasto, pero no solo el importe, sino también el concepto ("Café") y la categoría ("Desayuno"). Si metes todo eso en una lista normal, luego te será muy difícil saber qué es cada cosa.

Para estos casos, Python nos ofrece dos colecciones muy especiales: los **Diccionarios** y los **Conjuntos**.

◆ Diccionarios — datos con etiqueta

Un diccionario almacena pares clave-valor: en lugar de acceder a los datos por su posición como en una lista, accedes por su nombre. Es perfecto para organizar los gastos por categoría:

```
# Crear un diccionario de gastos por categoría
gastos_categoria = {
    "comida": 120.50,
    "transporte": 45.00,
    "ocio": 80.75,
    "casa": 350.00
}

# Acceder a un valor por su clave
print(gastos_categoria["comida"])    # 120.50

# Añadir o modificar una entrada
gastos_categoria["salud"] = 60.00
```

```
# Recorrer el diccionario
for categoria, total in gastos_categoria.items():
    print(f"{categoria}: {total} €")
```

Para saber más sobre Diccionarios, consulta [w3Schools](https://www.w3schools.com/python/python_dictionaries.asp). <https://www.w3schools.com/python/python_dictionaries.asp>

🍇 Conjuntos — sin repeticiones

Un conjunto almacena elementos únicos y sin orden. Su gran utilidad es eliminar duplicados automáticamente y realizar operaciones entre grupos. En el Gestor te sirve para obtener las categorías que has usado sin repetición:

```
# Categorías de todos los gastos registrados (con repetidos)
cats_registradas = ["comida", "ocio", "comida", "casa", "ocio", "comida"]

# Convertir a conjunto elimina los duplicados
cats_unicas = set(cats_registradas)
print(cats_unicas)    # {'comida', 'ocio', 'casa'}

# Operaciones entre conjuntos
presupuestadas = {"comida", "casa", "transporte", "salud"}
sin_gastar = presupuestadas - cats_unicas    # Categorías no usadas aún
print(sin_gastar)    # {'transporte', 'salud'}
```

Para saber más sobre Conjuntos, consulta [w3Schools](https://www.w3schools.com/python/python_sets.asp). <https://www.w3schools.com/python/python_sets.asp>

6. Practicamos sobre diccionarios y conjuntos

¿Cuál es la principal diferencia entre una lista y un diccionario?

- ☐ Las listas permiten duplicados y los diccionarios no.
- ☐ Incorrecto. En una lista se accede por posición y en un diccionario por clave
- ☐ Los diccionarios son inmutables como las tuplas
- ☐ Las listas solo almacenan números y los diccionarios texto

Tienes esta lista de categorías: ["comida","ocio","comida","casa","ocio"]. ¿Qué estructura usarías para obtener las categorías sin repetición?

- ☐ Una lista ordenada con sort()
- ☐ Un diccionario con las categorías como claves
- ☐ Un conjunto con set()
- ☐ Una tupla

Estás diseñando la estructura interna de tu Gestor de Gastos y necesitas guardar los datos del perfil del usuario: su nombre, su edad y su presupuesto mensual máximo. ¿Qué tipo de colección de datos sería la más lógica y profesional para guardar esta información junta?

- ☐ Una lista []
- ☐ Un conjunto { } (sin etiquetas)
- ☐ Un diccionario { } (con etiquetas)

Comprobar

7. El Gestor toma forma

Hasta ahora tu Gestor de Gastos pedía datos al usuario y los mostraba, pero no los guardaba. Ha llegado el momento de darle memoria. En esta actividad vas a construir una versión mejorada que almacena todos los gastos en una **lista**, lleva el total por categoría en un **diccionario** y usa una **tupla** con las categorías disponibles para validar la entrada del usuario.

Para ello, sigue los siguientes pasos:

1. Define una **tupla** con las categorías disponibles: ("comida", "transporte", "ocio", "casa", "salud")
2. Crea una **lista vacía** para almacenar los gastos y un diccionario vacío para el total por categoría.
3. Muestra un **menú con bucle while** que ofrezca estas opciones:

- **Opción 1** — Registrar gasto: pide importe y categoría. Si la categoría no está en la tupla, avisa al usuario y no registra el gasto. Si es válida, añade el gasto a la lista como diccionario {"importe": x, "categoría": y} y acumula el importe en el diccionario de totales.
- **Opción 2** — Ver todos los gastos: recorre la lista y muestra cada gasto con su importe y categoría.
- **Opción 3** — Ver resumen por categoría: recorre el diccionario de totales y muestra cuánto se ha gastado en cada categoría.
- **Opción 4** — Categorías usadas: usa un conjunto para mostrar qué categorías se han usado sin repetición.
- **Opción 5** — Salir.



- ♣ Para añadir un gasto como diccionario a la lista usa `gastos.append({"importe": importe, "categoría": categoría})`
- ♣ Para acumular en el diccionario de totales: `if categoría in totales: totales[categoría] += importe` y `else: totales[categoría] = importe`
- ♣ Para obtener las categorías usadas sin repetición: `set([g["categoría"] for g in gastos])`
- ♣ La salida de tu programa debería ser similar a esta:

```
=== MENÚ DEL GESTOR ===
Categorías disponibles: comida, transporte, ocio, casa, salud

1. Registrar gasto
2. Ver todos los gastos
3. Ver resumen por categoría
4. Ver categorías usadas
5. Salir

Elige una opción: 1
Importe: 45.50
Categoría: comida
Gasto registrado correctamente.

Elige una opción: 1
Importe: 30.00
Categoría: viajes
Categoría no válida. Elige entre: comida, transporte, ocio, casa, salud

Elige una opción: 2
Listado de gastos:
- comida: 45.50 €

Elige una opción: 3
Resumen por categoría:
- comida: 45.50 €

Elige una opción: 4
Categorías usadas: {'comida'}

Elige una opción: 5
¡Hasta pronto! Tu gestor ha terminado.
```

```
# El Gestor toma forma - Solución orientativa

CATEGORIAS = ("comida", "transporte", "ocio", "casa", "salud")

gastos = []          # Lista de gastos
totales = {}         # Diccionario de totales por categoría

while True:
    print("\n=== MENÚ DEL GESTOR ===")
    print(f"Categorías disponibles: {' '.join(CATEGORIAS)}")
```

```

print("\n1. Registrar gasto")
print("2. Ver todos los gastos")
print("3. Ver resumen por categoría")
print("4. Ver categorías usadas")
print("5. Salir")

opcion = input("\nElige una opción: ")

if opcion == "1":
    importe = float(input("Importe: "))
    categoria = input("Categoría: ").lower()

    if categoria not in CATEGORIAS:
        print(f" Categoría no válida. Elige entre: {'', '.join(CATEGORIAS)}")
    else:
        gastos.append({"importe": importe, "categoria": categoria})
        if categoria in totales:
            totales[categoria] += importe
        else:
            totales[categoria] = importe
        print(" Gasto registrado correctamente.")

elif opcion == "2":
    if not gastos:
        print(" No hay gastos registrados todavía.")
    else:
        print(" Listado de gastos:")
        for g in gastos:
            print(f" - {g['categoria']}: {g['importe']} €")

elif opcion == "3":
    if not totales:
        print(" No hay gastos registrados todavía.")
    else:
        print(" Resumen por categoría:")
        for cat, total in totales.items():
            print(f" - {cat}: {total:.2f} €")

elif opcion == "4":
    if not gastos:
        print(" No hay categorías usadas todavía.")
    else:
        usadas = set([g["categoria"] for g in gastos])
        print(f" Categorías usadas: {usadas}")

elif opcion == "5":
    print(" ¡Hasta pronto! Tu gestor ha terminado.")
    break

else:
    print(" Opción no válida. Elige entre 1 y 5.")

```

¿Tu programa no funciona? Comprueba esto:

- 🔦 ¿Has definido la tupla CATEGORIAS antes del bucle?
- 🔦 ¿Usas not in para validar la categoría antes de registrar?
- 🔦 ¿Compruebas si la categoría ya existe en totales antes de sumar?
- 🔦 ¿El break está dentro del elif opcion == "5"?
- 🔦 ¿Usas .lower() al leer la categoría para evitar problemas con mayúsculas?

Reto extra

Añade una opción 6 que muestre el gasto más alto registrado usando max() sobre la lista de gastos.



8. Importante

Listas []

Son colecciones ordenadas y modificables. Se crean con corchetes y nos permiten añadir elementos en cualquier momento utilizando funciones como .append().

Tuplas ()

Son colecciones ordenadas pero inmutables. Se crean con paréntesis y se utilizan para almacenar información que no debe ser alterada durante la ejecución del programa.

Las posiciones (índices)

¡Ojo a este detalle informático! Tanto en las listas como en las tuplas, el primer elemento no ocupa la posición 1, sino la posición 0. Si quieres ver el primer mes de tu tupla escribirías: `print(meses_año[0])`

Diccionarios {clave: valor}

Colecciones que asocian una etiqueta (clave) a una información (valor). Son mutables, por lo que puedes añadir nuevas etiquetas o modificar las existentes en cualquier momento.

Conjuntos {valor}

Colecciones desordenadas que tienen una regla de oro: todos sus elementos deben ser únicos. Si añades un dato que ya existe, el conjunto simplemente lo ignora.

Cuidado con las llaves

Tanto los diccionarios como los conjuntos utilizan las llaves {}. La diferencia visual radica en que el diccionario siempre lleva dos puntos (:) separando la clave del valor.

8. Divide y reutiliza

1. Introducción

Observa el código que llevas escrito hasta ahora en tu Gestor de Gastos. Probablemente ya tienes varias decenas de líneas y, si necesitas registrar un gasto en dos sitios distintos del programa, tienes que repetir el mismo código dos veces. Y si luego quieres cambiarlo, tendrás que modificarlo en cada lugar donde aparezca. Eso no es programar bien: es una fuente segura de errores.

Los programadores tienen un principio fundamental que se resume en tres letras: **DRY (Don't Repeat Yourself — No te repitas)**. La herramienta que hace posible este principio son las **funciones**: bloques de código con nombre propio que puedes escribir una vez y reutilizar todas las veces que necesites.

Pero no solo se trata de no repetirse. Las funciones también te permiten **dividir un problema grande en partes pequeñas** (divide y vencerás como vimos en la situación de aprendizaje anterior), cada una con una responsabilidad clara. Tu Gestor de Gastos dejará de ser un bloque enorme de código y pasará a estar organizado en piezas independientes, fáciles de leer, de modificar y de reutilizar:



Imagen de elaboración propia generada con Gemini <<https://gemini.google.com/>> .
Funciones vs sin funciones (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

Y cuando ya domines tus propias funciones, descubrirás que Python viene cargado de **bibliotecas**: colecciones de funciones escritas por otros programadores que puedes usar directamente en tu código sin tener que reinventar la rueda.


```
# Función sin retorno
# Llamamos a la función dos veces seguidas
mostrar_menu()
mostrar_menu()
```

También podemos mandarle datos a la función para que trabaje con ellos, son los **parámetros**, variables que ponemos dentro de los paréntesis.

Una función debería hacer **una sola cosa** y hacerla bien. Si tu función se llama registrar_gasto, solo debería registrar el gasto, no calcular totales ni mostrar menús. Esto se llama el **principio de responsabilidad única** y hace el código mucho más fácil de entender y mantener.

Cuando trabajas con funciones surge una pregunta importante: ¿pueden las funciones ver y modificar las variables que has creado fuera de ellas? La respuesta depende del ámbito (scope en inglés) de cada variable.

En Python existen dos tipos de variables según dónde se han creado:

- 📌 **Variable local:** creada dentro de una función. Solo existe mientras la función se ejecuta y desaparece cuando termina. No es accesible desde fuera.
- 📌 **Variable global:** creada fuera de cualquier función. Es accesible desde cualquier parte del programa, incluso desde dentro de las funciones, aunque solo para leerla.

```
presupuesto = 500      # Variable GLOBAL - accesible en todo el programa

def comprobar_presupuesto(total):
    margen = presupuesto - total  # Lee la variable global
    limite = 0.80                # Variable LOCAL - solo existe aquí
    if total > presupuesto * limite:
        print(f" Llevas más del 80%. Margen restante: {margen:.2f} €")

comprobar_presupuesto(420)
print(limite)           # ERROR: limite no existe fuera de la función
```



Por defecto no. Si intentas asignar un nuevo valor a una variable global dentro de una función, Python crea una variable local con el mismo nombre sin modificar la global. Para modificarla explícitamente debes usar la palabra reservada *global*.

```
total_mes = 0      # Variable global

def registrar_gasto(importe):
    global total_mes  # Indicamos que usamos la variable global
    total_mes += importe  # Ahora sí modifica la variable global

registrar_gasto(45.50)
registrar_gasto(32.00)
print(f"Total acumulado: {total_mes} €")  # 77.50 €
```

Usar la palabra reservada global funciona pero no es una buena práctica. Lo más recomendable es pasar la variable como parámetro y devolverla con return. Así el código es más limpio y predecible.

2. Repasamos lo aprendido

Responde todas las preguntas del cuestionario.

Una función en Python debe devolver siempre un valor con *return*.

[Sugerencia](#)

☐ Verdadero ☐ Falso

Puedes llamar a una función tantas veces como necesites a lo largo del programa.

[Sugerencia](#)

☐ Verdadero ☐ Falso

Los parámetros de una función son los valores que le pasas cuando la llamas.

[Sugerencia](#)

☐ Verdadero ☐ Falso

El nombre de la función puede ser cualquier palabra, incluidas las palabras reservadas de Python.

[Sugerencia](#)

☐ Verdadero ☐ Falso

Una función puede llamar a otra función dentro de su bloque de código.

[Sugerencia](#)

☐ Verdadero ☐ Falso

3. Refactoriza tu Gestor

Ha llegado el momento de dar un gran salto de calidad en tu Gestor de Gastos. Toma el programa que construiste en el bloque de estructuras de datos y **refactorízalo usando funciones**: divide el código en funciones con responsabilidad única, una por cada acción del menú.

Las funciones que debes crear:

```
def mostrar_menu(categorias):
    # Muestra el menú con las categorías disponibles

def registrar_gasto(gastos, totales, categorias):
    # Pide importe y categoría, valida y registra
    # Devuelve True si se registró, False si no

def mostrar_gastos(gastos):
    # Muestra todos los gastos de la lista

def mostrar_resumen(totales):
    # Muestra el total por categoría

def mostrar_categorias_usadas(gastos):
    # Muestra las categorías únicas usadas
```

El bucle principal debería quedar así de limpio:

```
while True:
    mostrar_menu(CATEGORIAS)
    opcion = input("Elige una opción: ")

    if opcion == "1":
        registrar_gasto(gastos, totales, CATEGORIAS)
    elif opcion == "2":
        mostrar_gastos(gastos)
    elif opcion == "3":
        mostrar_resumen(totales)
    elif opcion == "4":
        mostrar_categorias_usadas(gastos)
    elif opcion == "5":
        print(" ¡Hasta pronto!")
        break
    else:
        print(" Opción no válida.")
```

Gestor de Gastos – Versión con funciones

```
CATEGORIAS = ("comida", "transporte", "ocio", "casa", "salud")
```

```
def mostrar_menu(categorias):  
    print("\n=== MENÚ DEL GESTOR ===")  
    print(f"Categorías: {' '.join(categorias)}")  
    print("\n1. Registrar gasto")  
    print("2. Ver todos los gastos")  
    print("3. Ver resumen por categoría")  
    print("4. Ver categorías usadas")  
    print("5. Salir")
```

```
def registrar_gasto(gastos, totales, categorias):  
    importe = float(input("Importe: "))  
    categoria = input("Categoría: ").lower()  
    if categoria not in categorias:  
        print(f"Categoría no válida.")  
        return False  
    gastos.append({"importe": importe, "categoria": categoria})  
    totales[categoria] = totales.get(categoria, 0) + importe  
    print("Gasto registrado correctamente.")  
    return True
```

```
def mostrar_gastos(gastos):  
    if not gastos:  
        print(" No hay gastos registrados.")  
        return  
    print(" Listado de gastos:")  
    for g in gastos:  
        print(f" - {g['categoria']}: {g['importe']} €")
```

```
def mostrar_resumen(totales):  
    if not totales:  
        print(" No hay datos todavía.")  
        return  
    print(" Resumen por categoría:")  
    for cat, total in totales.items():  
        print(f" - {cat}: {total:.2f} €")
```

```
def mostrar_categorias_usadas(gastos):  
    if not gastos:  
        print(" No hay categorías usadas todavía.")  
        return  
    usadas = set([g["categoria"] for g in gastos])  
    print(f" Categorías usadas: {usadas}")
```

```
# Programa principal
```

```
gastos = []  
totales = {}
```

```
while True:  
    mostrar_menu(CATEGORIAS)  
    opcion = input("\nElige una opción: ")  
  
    if opcion == "1":  
        registrar_gasto(gastos, totales, CATEGORIAS)  
    elif opcion == "2":  
        mostrar_gastos(gastos)  
    elif opcion == "3":  
        mostrar_resumen(totales)  
    elif opcion == "4":  
        mostrar_categorias_usadas(gastos)  
    elif opcion == "5":  
        print(" ¡Hasta pronto!")  
        break  
    else:  
        print(" Opción no válida. Elige entre 1 y 5.")
```

¿Tu programa no funciona? Comprueba esto:

- ◆ ¿Cada función hace solo una cosa?
- ◆ ¿Pasas las variables como parámetros en lugar de usar global?
- ◆ ¿Las funciones que modifican datos reciben la lista o diccionario como parámetro?
- ◆ ¿El bucle principal llama a las funciones correctamente?

Reto extra

Crea una función `calcular_estadisticas(gastos)` que devuelva en un diccionario el total, la media, el gasto máximo y el mínimo del mes.

4. Importante

Definir funciones

def (definir), es la palabra clave para crear una función. Le enseña al ordenador cómo hacer una tarea, agrupando varias líneas de código bajo un solo nombre.

Invocar funciones

Para usar una función que ya existe, solo tienes que escribir su nombre seguido de paréntesis, por ejemplo: `mostrar_menu()`

Retorno de funciones

return (devolver), se utiliza dentro de la función cuando queremos que el resultado de un cálculo o proceso sea devuelto al programa principal para poder guardarlo en una variable y seguir trabajando con él.

Apoyo visual



Imagen de elaboración propia generada con ChatGPT <<https://chatgpt.com/>> . ¿Por qué utilizar funciones? (CC BY-NC-SA <<http://creativecommons.org/licenses/>>)

8.2 No reinventes la rueda

1. Librerías o bibliotecas



Imagen de elaboración propia generada con Gemini <<https://gemini.google.com/>> .
Librerías (CC BY-NC-SA <<http://creativecommons.org/licenses/>>)

Imagina que estás construyendo una casa. Puedes fabricar tus propios martillos y sierras (eso sería crear tus propias funciones con `def`), pero lo más lógico e inteligente es ir a la ferretería y comprar una caja de herramientas que alguien ya ha fabricado por ti.

En Python, a estas cajas de herramientas las llamamos **Bibliotecas** o **Librerías**. Son archivos repletos de funciones creadas por programadores expertos que puedes traer a tu programa para usarlas al instante.

Python viene "de fábrica" con una librería estándar gigantesca que te permite hacer cosas increíbles: desde operaciones matemáticas complejas hasta generar números aleatorios o conectarte a internet.

¿Cómo traigo la caja de herramientas a mi programa?

Para usar una librería, tienes que pedirle a Python que la importe (la traiga a tu archivo) utilizando la palabra clave `import`. Esta orden se pone siempre al principio del todo de tu código, en la primera línea.

Vamos a ver un ejemplo perfecto para tu Gestor de Gastos. Seguramente querrás que al introducir un ticket, el programa guarde automáticamente la fecha exacta de ese día. Para no tener que programar un calendario desde cero, usamos la librería `datetime` (fecha y hora):

```
# 1. Traemos la caja de herramientas de fechas al principio del programa
import datetime

print("--- REGISTRO DE GASTOS ---")
importe = float(input("Introduce el importe: "))

# 2. Usamos una herramienta de esa caja (usando el punto para abrirla)
fecha_hoy = datetime.date.today()

print("Gasto de", importe, "€ guardado correctamente con fecha:", fecha_hoy)
```

Fíjate en el paso 2: para usar una función que está dentro de una librería, primero escribimos el nombre de la librería, luego un punto (que significa "entra ahí y busca"), y luego el nombre de la función concreta.

Podemos también importar solo algunas funciones de una librería y no la librería completa:

```
import datetime      #Importa toda la biblioteca
import math          #Importa toda la biblioteca

from datetime import date      #Importa solo lo que necesitas
from math import sqrt, ceil    # Importa funciones concretas

import datetime as dt         #Importa con un alias más corto
```

Librería estándar vs librería de terceros

No todas las librerías funcionan igual. Antes de usarlas para tu Gestor, conviene que conozcas esta diferencia:

- Las bibliotecas de la **librería estándar** de Python (*datetime*, *csv*, *json*, *os* y *tkinter*) vienen incluidas con Python desde el momento de la instalación. Solo necesitas importarlas directamente en tu código, sin ningún paso previo.
- Las **librerías de terceros** como *matplotlib* han sido desarrolladas por programadores externos y no vienen incluidas con Python. Antes de importarlas debes instalarlas una sola vez usando *pip*, el gestor de paquetes de Python. En el terminal integrado de VS Code escribe:

`pip install matplotlib`

Una vez instalada, ya puedes importarla con normalidad en cualquier programa.

📌 2. Herramientas para tu Gestor

A continuación vamos a ver algunas librerías que te pueden resultar muy útiles para tu Gestor de gastos.

No te preocupes sino entiendes el código, pruébalo y observa el funcionamiento de cada librería.

📅 Datetime

Permite trabajar con fechas y horas. En tu Gestor la usarás para registrar automáticamente cuándo se realizó cada gasto, sin necesidad de pedirselo al usuario.

📌 Cómo importarla:

```
from datetime import date, datetime
```

📌 Algunas funciones útiles:

Función/atributo	Descripción	Ejemplo
<code>datetime.date.today()</code>	fecha actual	2026-05-21
<code>datetime.datetime.now()</code>	Fecha y hora actual	2026-05-21 19:15:05
<code>.strftime("%d/%m/%Y")</code>	Formatea una fecha como texto	"21/05/2026"

<code>.strftime("15/11/2024", "%d/%m/%Y")</code>	Convierte texto a fecha	<code>date(2026, 05, 21)</code>
<code>datetime.timedelta(days=30)</code>	Diferencia entre fechas	Sumar o restar días
<code>.day / .month / .year</code>	Extrae día, mes o año	21 / 05 / 2026

📌 Ejemplo en el Gestor:

```
import datetime

# Fecha y hora actuales
hoy = datetime.date.today()
ahora = datetime.datetime.now()

print("=== INFORMACIÓN DE FECHA Y HORA ===")
print(f"Fecha de hoy: {hoy.strftime('%d/%m/%Y')}")
print(f"Hora actual: {ahora.strftime('%H:%M:%S')}")
print(f"Día de la semana: {hoy.strftime('%A')}")
print(f"Mes actual: {hoy.strftime('%B')}")

# Días transcurridos desde el inicio del mes
inicio_mes = datetime.date(hoy.year, hoy.month, 1)
dias_transcurridos = (hoy - inicio_mes).days + 1
dias_restantes = 30 - dias_transcurridos

print(f"\n=== CONTROL DEL MES ===")
print(f"Llevas {dias_transcurridos} días de mes")
print(f"Quedan aproximadamente {dias_restantes} días para cerrar el mes")

# Cómo quedaría el registro de un gasto con fecha automática
print(f"\n=== REGISTRO DE GASTO DE EJEMPLO ===")
gasto = {
    "importe": 45.50,
    "categoria": "comida",
    "fecha": hoy.strftime("%d/%m/%Y"),
    "hora": ahora.strftime("%H:%M")
}
print(f"Gasto registrado:")
for clave, valor in gasto.items():
    print(f" {clave}: {valor}")
```

📌 Salida esperada:

```
=== INFORMACIÓN DE FECHA Y HORA ===
Fecha de hoy: 15/11/2024
Hora actual: 10:32:05
Día de la semana: Friday
Mes actual: November

=== CONTROL DEL MES ===
Llevas 15 días de mes
Quedan aproximadamente 15 días para cerrar el mes

=== REGISTRO DE GASTO DE EJEMPLO ===
Gasto registrado:
importe: 45.5
categoria: comida
fecha: 15/11/2024
hora: 10:32
```

- 📌 `strftime("%d/%m/%Y")` da formato a la fecha: día/mes/año. Puedes cambiar el formato a tu gusto: "%Y-%m-%d" para año-mes-día.
- 📌 Fíjate que no se importa el módulo `datetime` completo, sino solo la clase `date`.
- 📌 Documentación oficial: [Librería datetime](#)

📌 Matplotlib

Es la biblioteca de visualización más popular de Python. Genera gráficas de barras, sectores, líneas y muchas más. En tu Gestor la usarás para representar visualmente los gastos por categoría.

📌 Cómo instalarla e importarla:

```
# Primero instala en el terminal:
# pip install matplotlib

import matplotlib.pyplot as plt
```

📌 Algunas funciones útiles:

Función	Descripción	Ejemplo
plt.bar(x, y)	Gráfica de barras	Gastos por categoría
plt.pie(valores, labels=etiquetas)	Gráfica de sectores	Porcentaje por categoría
plt.plot(x, y)	Gráfica de líneas	Evolución de gastos
plt.title("texto")	Título de la gráfica	"Gastos de noviembre"
plt.xlabel("texto")	Etiqueta eje X	"Categoría"
plt.ylabel("texto")	Etiqueta eje Y	"Total (€)"
plt.figure(figsize=(8,5))	Tamaño de la gráfica	Ancho x alto en pulgadas
plt.tight_layout()	Ajusta márgenes	Evita recortes
plt.show()	Muestra la gráfica	Abre la ventana
plt.savefig("grafica.png")	Guarda la gráfica	Exporta a imagen

📌 Ejemplo en el Gestor:

```
import matplotlib.pyplot as plt

# Datos de ejemplo del Gestor de Gastos
totales = {
    "comida": 245.50,
    "transporte": 89.00,
    "ocio": 120.75,
    "casa": 450.00,
    "salud": 60.00
}

colores = ["#0891B2", "#7C3AED", "#D97706", "#059669", "#DC2626"]

categorias = list(totales.keys())
importes = list(totales.values())
total_mes = sum(importes)

# Crear figura con dos gráficas lado a lado
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle(" Resumen de gastos – Noviembre 2024",
             fontsize=14, fontweight="bold")

# Gráfica de barras
ax1.bar(categorias, importes, color=colores, width=0.6)
ax1.set_title("Gasto total por categoría")
ax1.set_xlabel("Categoría")
ax1.set_ylabel("Total (€)")
ax1.set_ylim(0, max(importes) * 1.15)

# Añadir valor encima de cada barra
for i, (cat, imp) in enumerate(zip(categorias, importes)):
    ax1.text(i, imp + 5, f"{imp:.0f} €",
             ha="center", fontsize=9, fontweight="bold")

# Gráfica de sectores
wedges, texts, autotexts = ax2.pie(
    importes,
    labels=categorias,
    autopct="%1.1f%%",
    colors=colores,
    startangle=90,
    pctdistance=0.75
)
ax2.set_title("Distribución por categoría")

# Añadir total en el centro
ax2.text(0, 0, f"Total\n{total_mes:.0f} €",
        ha="center", va="center",
        fontsize=11, fontweight="bold")

plt.tight_layout()
plt.savefig("resumen_gastos.png") # Guarda la imagen
plt.show()

print(f" Gráfica guardada como resumen_gastos.png")
print(f"\n Resumen numérico:")
for cat, imp in totales.items():
    porcentaje = (imp / total_mes) * 100
```

```
print(f" {cat:<15} {imp:>8.2f} € ({porcentaje:.1f}%)")
print(f" {'TOTAL':<15} {total_mes:>8.2f} € (100%)")
```

Salida esperada:

Gráfica guardada como resumen_gastos.png

Resumen numérico:

comida	245.50 €	(25.8%)
transporte	89.00 €	(9.3%)
ocio	120.75 €	(12.7%)
casa	450.00 €	(47.2%)
salud	60.00 €	(6.3%)
TOTAL	965.25 €	(100%)

Documentación oficial: [Librería Matplotlib <https://matplotlib.org/>](https://matplotlib.org/)

CSV

Permite leer y escribir archivos CSV (valores separados por comas). Es el formato más universal para intercambiar datos: cualquier hoja de cálculo como Excel o Google Sheets puede abrirlo directamente. Con ella tu Gestor podrá exportar e importar el historial de gastos.

Como importarla:

```
import csv
```

Algunas funciones útiles:

Función	Descripción	Uso en el gestor
csv.DictWriter(f, fieldnames)	Escribe filas como diccionarios	Exportar gastos
writer.writeheader()	Escribe la cabecera con los nombres de columnas	Primera fila del CSV
writer.writerow(dict)	Escribe una fila	Un gasto
writer.writerows(lista)	Escribe varias filas de golpe	Todos los gastos
csv.DictReader(f)	Lee filas como diccionarios	Importar gastos
csv.writer(f)	Escribe filas como listas	Formato simple
csv.reader(f)	Lee filas como listas	Lectura simple

Ejemplo en el gestor:

```
import csv
import os

# Datos de ejemplo del Gestor de Gastos
gastos = [
    {"fecha": "01/11/2024", "categoria": "comida", "importe": 45.50},
    {"fecha": "03/11/2024", "categoria": "transporte", "importe": 32.00},
    {"fecha": "05/11/2024", "categoria": "ocio", "importe": 18.75},
    {"fecha": "08/11/2024", "categoria": "casa", "importe": 120.00},
    {"fecha": "10/11/2024", "categoria": "salud", "importe": 9.99},
    {"fecha": "12/11/2024", "categoria": "comida", "importe": 67.30},
    {"fecha": "14/11/2024", "categoria": "ocio", "importe": 45.00},
]

# — PASO 1: Exportar gastos a CSV —
archivo = "gastos_noviembre.csv"
campos = ["fecha", "categoria", "importe"]

with open(archivo, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=campos)
    writer.writeheader()
    writer.writerows(gastos)

print(f"Archivo '{archivo}' creado con {len(gastos)} gastos")
print(f"    Tamaño: {os.path.getsize(archivo)} bytes")
print(f"    Puedes abrirlo con Excel o Google Sheets\n")

# — PASO 2: Importar y mostrar los gastos —
print("Contenido del archivo:")
print(f"{'FECHA':<14} {'CATEGORIA':<14} {'IMPORTE':>10}")
print("-" * 42)

total = 0.0
```

```
gastos_importados = []

with open(archivo, "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for fila in reader:
        fila["importe"] = float(fila["importe"])
        gastos_importados.append(fila)
        total += fila["importe"]
        print(f"{fila['fecha']:<14} {fila['categoria']:<14} "
              f"{fila['importe']:>9.2f} €")

print("-" * 42)
print(f"{'TOTAL':<28} {total:>9.2f} €")

# --- PASO 3: Resumen por categoría ---
print("\n Resumen por categoría:")
resumen = {}
for g in gastos_importados:
    cat = g["categoria"]
    resumen[cat] = resumen.get(cat, 0) + g["importe"]

for cat, subtotal in sorted(resumen.items(),
                           key=lambda x: x[1], reverse=True):
    porcentaje = (subtotal / total) * 100
    print(f" {cat:<14} {subtotal:>8.2f} € ({porcentaje:.1f}%")
```

Salida esperada:

Archivo 'gastos_noviembre.csv' creado con 7 gastos
 Tamaño: 248 bytes
 Puedes abrirlo con Excel o Google Sheets

Contenido del archivo:

FECHA	CATEGORÍA	IMPORTE
01/11/2024	comida	45.50 €
03/11/2024	transporte	32.00 €
05/11/2024	ocio	18.75 €
08/11/2024	casa	120.00 €
10/11/2024	salud	9.99 €
12/11/2024	comida	67.30 €
14/11/2024	ocio	45.00 €
TOTAL		338.54 €

Resumen por categoría:

casa	120.00 €	(35.4%)
comida	112.80 €	(33.3%)
ocio	63.75 €	(18.8%)
transporte	32.00 €	(9.5%)
salud	9.99 €	(2.9%)

Documentación oficial Librería CSV <<https://docs.python.org/es/3/library/csv.html>>

JSON

Permite guardar y cargar datos en formato JSON, conservando perfectamente la estructura de diccionarios y listas de Python. Es ideal para guardar el estado completo del Gestor entre sesiones, incluyendo los totales por categoría ya calculados.

¿Qué es JSON?

Imagina que tienes una libreta donde apuntas tus gastos con un formato muy concreto: primero la fecha, luego la categoría y después el importe. Ese formato ordenado es lo que permite que otra persona (o programa) lea tus anotaciones y las entienda sin problema.

JSON es exactamente eso: un formato estándar para guardar información de forma ordenada en un archivo de texto. Ni más ni menos.

Cómo importarla:

```
import json
```

Algunas funciones útiles:

Función	Descripción	Uso en el gestor
json.dump(datos, fichero)	Guarda datos en un archivo JSON	Guardar gastos y totales
json.load(fichero)	Carga datos desde un archivo JSON	Recuperar sesión anterior
json.dumps(datos)	Convierte datos a texto JSON	Mostrar datos como texto
json.loads(texto)	Convierte texto JSON a datos Python	Procesar JSON recibido

indent=2	Parámetro para formato legible	JSON con sangría
ensure_ascii=False	Permite tildes y ñ	Textos en español

📌 Ejemplo en el gestor:

```
import json

# Datos de ejemplo del Gestor de Gastos
datos = {
    "presupuesto": 500.00,
    "gastos": [
        {"fecha": "01/11/2024", "categoria": "comida", "importe": 45.50},
        {"fecha": "03/11/2024", "categoria": "transporte", "importe": 32.00},
        {"fecha": "05/11/2024", "categoria": "ocio", "importe": 18.75},
    ],
    "totales": {"comida": 45.50, "transporte": 32.00, "ocio": 18.75}
}

# --- PASO 1: Guardar en archivo JSON ---
with open("gestor_demo.json", "w", encoding="utf-8") as f:
    json.dump(datos, f, ensure_ascii=False, indent=2)
print(" Datos guardados en gestor_demo.json\n")

# --- PASO 2: Mostrar cómo queda el archivo ---
print(" 📄 Contenido del archivo:")
with open("gestor_demo.json", "r", encoding="utf-8") as f:
    print(f.read())

# --- PASO 3: Cargar y usar los datos ---
with open("gestor_demo.json", "r", encoding="utf-8") as f:
    datos_cargados = json.load(f)

total = sum(datos_cargados["totales"].values())
print(f" Total gastado: {total:.2f} €")
print(f" Disponible: {datos_cargados['presupuesto'] - total:.2f} €")
```

📌 Salida esperada:

```
Datos guardados en gestor_demo.json

Contenido del archivo:
{
  "presupuesto": 500.0,
  "gastos": [...],
  "totales": {
    "comida": 45.5,
    "transporte": 32.0,
    "ocio": 18.75
  }
}

Total gastado: 96.25 €
Disponible: 403.75 €
```

📌 Documentación oficial: [Librería JSON](#)

📌 Tkinter

Es la biblioteca gráfica incluida en Python. Permite crear ventanas reales con botones, campos de texto, listas y menús. Con ella tu Gestor dejará de ser un programa de terminal y tendrá una interfaz gráfica con ventanas, como cualquier aplicación profesional.

📌 Como importarla:

```
import tkinter as tk
from tkinter import messagebox
```

📌 Algunas funciones útiles:

Componente/Función	Descripción	Uso en el Gestor
tk.Tk()	Crea la ventana principal	Ventana del Gestor
tk.Label(ventana, text)	Muestra texto estático	Títulos y etiquetas
tk.Entry(ventana)	Campo de texto editable	Introducir importe

tk.Button(ventana, text, command)	Botón clicable	Registrar gasto
tk.Listbox(ventana)	Lista de elementos	Mostrar gastos
messagebox.showinfo(titulo, msg)	Ventana de información	Confirmar registro

📌 Ejemplo en el Gestor:

```
import tkinter as tk
from tkinter import messagebox

gastos = []
total = 0.0

def registrar():
    global total
    try:
        importe = float(entrada_importe.get())
        categoria = entrada_cat.get().lower().strip()
    except ValueError:
        messagebox.showerror("Error", "El importe debe ser un número.")
        return

    gastos.append({"importe": importe, "categoria": categoria})
    total += importe

    lista.insert(tk.END, f" {categoria:<14} {importe:.2f} €")
    etiqueta_total.config(text=f"Total: {total:.2f} €")
    entrada_importe.delete(0, tk.END)
    entrada_cat.delete(0, tk.END)
    entrada_importe.focus()

# Ventana principal
ventana = tk.Tk()
ventana.title(" Gestor de Gastos")
ventana.geometry("380x400")
ventana.resizable(False, False)

tk.Label(ventana, text=" Gestor de Gastos",
        font=("Arial", 14, "bold")).pack(pady=10)

tk.Label(ventana, text="Importe (€):").pack()
entrada_importe = tk.Entry(ventana, width=25)
entrada_importe.pack(pady=4)

tk.Label(ventana, text="Categoría:").pack()
entrada_cat = tk.Entry(ventana, width=25)
entrada_cat.pack(pady=4)

tk.Button(ventana, text=" Registrar gasto",
        command=registrar,
        bg="#0891B2", fg="white",
        font=("Arial", 11, "bold")).pack(pady=10)

lista = tk.Listbox(ventana, width=40, height=8,
        font=("Courier", 10))
lista.pack(padx=20)

etiqueta_total = tk.Label(ventana,
        text="Total: 0.00 €",
        font=("Arial", 12, "bold"), fg="#059669")
etiqueta_total.pack(pady=8)

ventana.mainloop()
```

📌 Salida esperada:



Imagen de elaboración propia. Ventana de Gastos (CC BY-NC-SA
<<http://creativecommons.org/licenses/>>)

Al ejecutarlo se abrirá una ventana funcional: introduce un importe y una categoría, pulsa el botón y verás cómo se añade a la lista y el total se actualiza automáticamente.

📌 Guía de tkinter en español: [Librería Tkinter](https://docs.python.org/es/3/library/tkinter.html) <<https://docs.python.org/es/3/library/tkinter.html>>

👉 3. Practicamos lo aprendido

¿Qué biblioteca de Python usarías en cada situación? Elige la respuesta correcta.

5 ✓ 0 ✗ 0 🏆 0

🕒 00:00 🔄 📄



[Pulse aquí para jugar](#)

📍 4. Importante

📖 Librerías

Son colecciones de funciones y código ya escrito por otros programadores que resuelven problemas comunes. Nos ahorran muchísimo tiempo de desarrollo.

📖 La orden import

Es la llave para usar una biblioteca. Se coloca siempre en la cabecera de nuestro archivo (en las primeras líneas). Ejemplo: `import math`.

📖 Sintaxis

Para utilizar una función específica que vive dentro de una biblioteca que has importado, escribes el nombre de la biblioteca, seguido de un punto y el nombre de la función. Ejemplo: `math.sqrt(9)` (para calcular la raíz cuadrada).

9. La memoria a largo plazo

1. Los archivos

Imagina que pasas horas introduciendo todos los tickets del mes en tu Gestor de Gastos. Estás orgulloso de tu trabajo, pero al cerrar VSCode... ¡puf! Todo desaparece. Esto ocurre porque las variables (como tus listas y diccionarios) viven en la memoria RAM del ordenador, que se borra al apagar el programa.

Cuando un programa termina de ejecutarse, todo lo que había en sus variables desaparece de la memoria del ordenador. Es como si cada vez que cerraras una libreta, todas las anotaciones se borrasen solas. Evidentemente, un gestor de gastos que no recuerda nada no sirve de mucho.

Para que tu aplicación tenga "memoria a largo plazo", necesitamos que escriba esos datos en un archivo físico (como un .txt o un .csv) guardado en tu disco duro. En Python, hacer esto es increíblemente fácil.



Imagen de elaboración propia generada con Gemini <<https://gemini.google.com/>> .
Persistencia de datos (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

2. Leer y escribir archivos

Trabajar con archivos en programación es exactamente igual que trabajar con una libreta de papel en la vida real. Tienes que seguir tres pasos:

1. Abrir la libreta.
2. Leer lo que pone o escribir algo nuevo.
3. Cerrar la libreta y guardarla en el cajón.

En Python trabajar con archivos es muy sencillo. Todo gira en torno a la función `open()`, que abre un archivo y te devuelve un objeto con el que puedes leer o escribir. Cuando terminas, debes cerrarlo para liberar recursos.

La función `open()` y los modos de apertura

```
archivo = open("gastos.txt", "r") # Abre para leer
archivo = open("gastos.txt", "w") # Abre para escribir (borra el contenido anterior)
archivo = open("gastos.txt", "a") # Abre para añadir al final
archivo = open("gastos.txt", "x") # Crea un archivo nuevo (error si ya existe)
```



La magia de la palabra `with`

El error más común de los programadores principiantes es olvidar "cerrar la libreta" (usar `.close()`), lo que puede corromper el archivo. Para evitarlo, Python inventó una estructura maravillosa usando la palabra `with` (con).

Al usar `with`, Python se encarga de abrir el archivo, dejarte escribir, y cerrarlo automáticamente en cuanto terminas.

```
# ❌ Sin with: hay que cerrar manualmente
archivo = open("gastos.txt", "w")
archivo.write("45.50,comida,01/11/2024\n")
archivo.close() # Si olvidamos esto, el archivo puede corromperse

# ✅ Con with: se cierra automáticamente
with open("gastos.txt", "w", encoding="utf-8") as archivo:
    archivo.write("45.50,comida,01/11/2024\n")
    archivo.write("32.00,transporte,03/11/2024\n")
```

Usa siempre *with* para abrir archivos. Es más seguro, más limpio y es la práctica estándar en Python.

Leer

```
# Leer todo el contenido de una vez
with open("gastos.txt", "r", encoding="utf-8") as archivo:
    contenido = archivo.read()
    print(contenido)

# Leer línea a línea (útil para archivos grandes)
with open("gastos.txt", "r", encoding="utf-8") as archivo:
    for linea in archivo:
        print(linea.strip()) # strip() elimina el salto de línea

# Leer todas las líneas en una lista
with open("gastos.txt", "r", encoding="utf-8") as archivo:
    lineas = archivo.readlines()
    print(f"El archivo tiene {len(lineas)} gastos registrados")
```

Escribir

```
gastos = [
    "45.50,comida,01/11/2024",
    "32.00,transporte,03/11/2024",
    "18.75,ocio,05/11/2024"
]

# Escribir varias líneas de golpe
with open("gastos.txt", "w", encoding="utf-8") as archivo:
    archivo.writelines(f"{gasto}\n" for gasto in gastos)

# Añadir un gasto sin borrar los anteriores
with open("gastos.txt", "a", encoding="utf-8") as archivo:
    archivo.write("120.00,casa,08/11/2024\n")
    print(" Gasto añadido al historial")
```

3. Relaciona

Relaciona el modo con su uso:

5

Comprobar



"r"

Abre para leer. Error si el archivo no existe

"w"

Crea un archivo nuevo. Error si ya existe

"a"

Abre para añadir datos al final sin borrar nada

with open(...) as f:

"X"

- Abre para escribir. Borra el contenido anterior
- Abre y cierra el archivo automáticamente

📌 4. El Gestor con memoria

Ya sabes abrir, leer y escribir archivos. Ahora vamos a aplicarlo directamente al Gestor de Gastos para que recuerde los datos entre sesiones. Usaremos *json* para guardar los datos, *os* para comprobar si el archivo existe y *try/except* para gestionar los posibles errores.

📁 os — El guardián de los archivos

Antes de intentar abrir un archivo, es fundamental comprobar que existe y que está en condiciones de ser leído. Para eso usamos la biblioteca *os*, incluida en Python sin necesidad de instalarla:

```
import os
```

Las funciones de *os* más útiles para trabajar con archivos en el Gestor son:

Haz clic sobre la imagen para ampliar

Biblioteca os — Funciones para archivos		
Herramientas para gestionar archivos y carpetas desde Python		
Función	Descripción	Ejemplo en el Gestor
<code>os.path.exists("ruta")</code>	¿Existe el archivo o carpeta?	<code>os.path.exists("gestor.json")</code>
<code>os.path.getsize("ruta")</code>	Tamaño del archivo en bytes	<code>os.path.getsize("gestor.json")</code>
<code>os.getcwd()</code>	Carpeta de trabajo actual	<code>print(os.getcwd())</code>
<code>os.mkdir("carpeta")</code>	Crea una carpeta nueva	<code>os.mkdir("backlog")</code>
<code>os.remove("ruta")</code>	Elimina un archivo	<code>os.remove("gestor_viejo.json")</code>
<code>os.path.join("dir", "file")</code>	Une rutas de forma segura	<code>os.path.join("datos", "gestor.json")</code>

⚠️ Úsalas siempre ANTES de abrir un archivo para evitar errores inesperados

Imagen de elaboración propia generada con Claude <<https://claude.ai/>>. Tabla de funciones *os* (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

Aplicadas al Gestor, así es como las usarías antes de intentar cargar los datos:

```
import os

ARCHIVO = "gestor.json"

# ¿Existe el archivo?
if not os.path.exists(ARCHIVO):
    print(" No hay datos previos. Empezando desde cero.")

# ¿Está vacío?
elif os.path.getsize(ARCHIVO) == 0:
    print(" El archivo existe pero está vacío.")

# Todo correcto, podemos cargarlo
else:
    tamaño = os.path.getsize(ARCHIVO)
    print(f" Archivo encontrado ({tamaño} bytes). Cargando datos..." )
```

Combinar *os.path.exists()* con *os.path.getsize()* antes de abrir un archivo es una buena práctica que evita errores inesperados, especialmente en la primera ejecución del programa o si el archivo se ha corrompido.

📁 El manejo de excepciones: try / except

Cuando trabajas con archivos siempre pueden ocurrir errores imprevistos: el archivo no existe, está corrupto o no tienes permisos para abrirlo. Si no los gestionas, tu programa se cerrará con un error feo. La solución es el bloque *try/except*:

```
try:
    with open("gestor.json", "r", encoding="utf-8") as f:
        datos = json.load(f)
        print("Datos cargados correctamente")

except FileNotFoundError:
```

```

    print(" No hay datos previos. Empezando desde cero.")
    datos = {"gastos": [], "totales": {}}

except json.JSONDecodeError:
    print(" El archivo está dañado. Empezando desde cero.")
    datos = {"gastos": [], "totales": {}}

```

`try` contiene el código que puede fallar. `except` captura el error específico y lo gestiona sin que el programa se cierre. Puedes encadenar varios `except` para distintos tipos de error.



Con todo lo aprendido, el Gestor completo con memoria quedaría así:

```

import json
import os

# Nombre del archivo donde se guardarán los datos del gestor
ARCHIVO = "gestor.json"

def cargar_datos():
    """Carga los datos del archivo si existe y está en buen estado."""

    # Comprobamos si el archivo existe antes de intentar abrirlo
    if not os.path.exists(ARCHIVO):
        print(" No hay datos previos. Empezando desde cero.")
        return [], {} # Devolvemos estructuras vacías para empezar desde cero

    if os.path.getsize(ARCHIVO) == 0: # Comprobamos que el archivo no esté vacío
        print("El archivo está vacío. Empezando desde cero.")
        return [], {}

    try:
        with open(ARCHIVO, "r", encoding="utf-8") as f: # Abrimos el archivo en modo lectura
            datos = json.load(f) # Convertimos el contenido JSON a estructuras Python
            print("Sesión anterior cargada correctamente.")
            return datos["gastos"], datos["totales"] # Devolvemos la lista de gastos y el diccionario de totales
    except json.JSONDecodeError:
        print("El archivo está dañado. Empezando desde cero.") # El archivo existe pero su contenido no es JSON válido
        return [], {}
    except KeyError:
        print("Formato incorrecto. Empezando desde cero.") # El archivo JSON no tiene la estructura esperada
        return [], {}

def guardar_datos(gastos, totales):
    """Guarda los datos actuales en el archivo JSON."""

    try:
        # Creamos el diccionario con todos los datos a guardar
        datos = {
            "gastos": gastos, # Lista completa de gastos
            "totales": totales # Diccionario de totales por categoría
        }
        # Abrimos el archivo en modo escritura (sobrescribe el contenido anterior)
        with open(ARCHIVO, "w", encoding="utf-8") as f:
            # Guardamos los datos en formato JSON legible (indent=2)
            # ensure_ascii=False permite guardar tildes y ñ correctamente
            json.dump(datos, f, ensure_ascii=False, indent=2)

        print("Datos guardados correctamente.")

    except IOError:
        # Error de entrada/salida: permisos, disco lleno, etc.
        print("Error al guardar. Comprueba los permisos del archivo.")

```



5. Revisa lo aprendido

Elija la respuesta correcta

[Pulse aquí para jugar](#)

6. Dale memoria a tu Gestor

Integra las funciones `cargar_datos()` y `guardar_datos()` en el Gestor de Gastos que construiste en el bloque anterior. El programa debe:

1. **Al arrancar:** cargar los datos del archivo `gestor.json` si existe.
2. **Al registrar un gasto:** guardar automáticamente los datos actualizados.
3. **Al salir:** confirmar que los datos han quedado guardados.

Pista: Llama a `cargar_datos()` antes del bucle `while` y a `guardar_datos()` dentro de la opción 1 (registrar gasto) y también en la opción 5 (salir).



Revisa solución

```
import json
import os

CATEGORIAS = ("comida", "transporte", "ocio", "casa", "salud")
ARCHIVO = "gestor.json"

def cargar_datos():
    if not os.path.exists(ARCHIVO):
        print("❌ No hay datos previos. Empezando desde cero.")
        return [], {}
    try:
        with open(ARCHIVO, "r", encoding="utf-8") as f:
            datos = json.load(f)
            print(f"✅ Sesión cargada: {len(datos['gastos'])} gastos registrados.")
            return datos["gastos"], datos["totales"]
    except (json.JSONDecodeError, KeyError):
        print("⚠️ Archivo dañado. Empezando desde cero.")
        return [], {}

def guardar_datos(gastos, totales):
    try:
        with open(ARCHIVO, "w", encoding="utf-8") as f:
            json.dump({"gastos": gastos, "totales": totales},
                      f, ensure_ascii=False, indent=2)
            print("✅ Datos guardados correctamente.")
    except IOError:
        print("❌ Error al guardar.")

def mostrar_menu():
    print("\n=== MENÚ DEL GESTOR ===")
    print("1. Registrar gasto")
    print("2. Ver todos los gastos")
    print("3. Ver resumen por categoría")
    print("4. Ver categorías usadas")
    print("5. Salir")

def registrar_gasto(gastos, totales):
    importe = float(input("Importe: "))
    categoria = input("Categoría: ").lower().strip()
    if categoria not in CATEGORIAS:
        print(f"⚠️ Categoría no válida.")
        return
    import datetime
    fecha = datetime.date.today().strftime("%d/%m/%Y")
    gastos.append({"importe": importe,
                  "categoria": categoria, "fecha": fecha})
```

```

    totales[categoria] = totales.get(categoria, 0) + importe
    guardar_datos(gastos, totales)
    print(f"✅ Gasto de {importe} € en {categoria} registrado.")

def mostrar_gastos(gastos):
    if not gastos:
        print("📄 No hay gastos registrados.")
        return
    print("📄 Listado de gastos:")
    for g in gastos:
        print(f"    {g['fecha']}    {g['categoria']:<14} {g['importe']:.2f} €")

def mostrar_resumen(totales):
    if not totales:
        print("📊 No hay datos todavía.")
        return
    print("📊 Resumen por categoría:")
    for cat, total in totales.items():
        print(f"    {cat:<14} {total:.2f} €")

def mostrar_categorias_usadas(gastos):
    if not gastos:
        print("👉 No hay categorías usadas.")
        return
    usadas = set(g["categoria"] for g in gastos)
    print(f"👉 Categorías usadas: {usadas}")

# --- Programa principal ---
gastos, totales = cargar_datos()

while True:
    mostrar_menu()
    opcion = input("\nElige una opción: ")

    if opcion == "1":
        registrar_gasto(gastos, totales)
    elif opcion == "2":
        mostrar_gastos(gastos)
    elif opcion == "3":
        mostrar_resumen(totales)
    elif opcion == "4":
        mostrar_categorias_usadas(gastos)
    elif opcion == "5":
        guardar_datos(gastos, totales)
        print("👉 ¡Hasta pronto!")
        break
    else:
        print("⚠️ Opción no válida. Elige entre 1 y 5.")

```

¿Tu programa no funciona? Comprueba esto:

¿Llamas a `cargar_datos()` antes del bucle `while`?

¿Guardas los datos después de registrar un gasto?

¿Usas `try/except` para gestionar errores al cargar?

¿El archivo `gestor.json` se crea en la misma carpeta que tu programa?



7. Importante

Apertura de archivos

Abre siempre los archivos con `with`. El gestor de contexto `with open(...)` as `f`: cierra el archivo automáticamente aunque ocurra un error. Es la forma correcta y segura de trabajar con archivos en Python.

```

# Correcto
with open("gestor.json", "r", encoding="utf-8") as f:
    datos = json.load(f)

# Incorrecto - hay que cerrar manualmente
f = open("gestor.json", "r")
datos = json.load(f)
f.close()

```

Modos de apertura

El modo que uses al abrir un archivo determina qué puedes hacer con él y qué ocurre con su contenido anterior.

Modo	Acción	¿Borra el contenido?
"r"	Leer	No
"w"	Escribir desde cero	Sí
"a"	Añadir al final	No
"x"	Crear nuevo	No existe aún

El error más frecuente es usar "w" creyendo que añade datos cuando en realidad borra todo lo anterior. Para añadir sin borrar usa siempre "a".

Uso de os

Usa *os* antes de abrir un archivo. Comprueba siempre si el archivo existe antes de intentar cargarlo. Evitarás errores en la primera ejecución del programa.

```
import os

if not os.path.exists("gestor.json"):
    print(" No hay datos previos. Empezando desde cero.")
    gastos, totales = [], {}
else:
    # El archivo existe, podemos cargarlo
    with open("gestor.json", "r", encoding="utf-8") as f:
        datos = json.load(f)
```

Guarda datos con JSON

Guarda tus datos con *json*.

- 🔹 *json.dump()* guarda listas y diccionarios conservando su estructura.
- 🔹 *json.load()* los recupera exactamente igual.

Añade siempre `ensure_ascii=False` para que las tildes y la ñ se guarden correctamente.

```
import json

# Guardar
with open("gestor.json", "w", encoding="utf-8") as f:
    json.dump({"gastos": gastos, "totales": totales},
              f, ensure_ascii=False, indent=2)

# Cargar
with open("gestor.json", "r", encoding="utf-8") as f:
    datos = json.load(f)
    gastos = datos["gastos"]
    totales = datos["totales"]
```

Gestiona los errores con try/except

Cuando trabajas con archivos siempre pueden ocurrir errores imprevistos. Captura al menos *FileNotFoundError* y *json.JSONDecodeError* para que tu programa no se cierre bruscamente.

```
try:
    with open("gestor.json", "r", encoding="utf-8") as f:
        datos = json.load(f)

except FileNotFoundError:
    print("No hay datos previos.")
    datos = {"gastos": [], "totales": {}}

except json.JSONDecodeError:
    print(" Archivo dañado. Empezando desde cero.")
    datos = {"gastos": [], "totales": {}}
```

10. El mundo en objetos

1. Introducción

Cuando empezaste a programar en este curso, escribías instrucciones una detrás de otra: primero pedías el importe, luego lo guardabas en una variable, luego lo sumabas al total. Eso es programación **secuencial**. Después aprendiste a agrupar instrucciones en funciones reutilizables. Eso es programación **estructurada o procedural**. Ambos enfoques funcionan, pero tienen un límite: cuando los programas crecen y se complican, el código se vuelve difícil de organizar, de entender y de mantener.

Ha llegado el momento de dar el salto definitivo en tu forma de programar y acercarte a cómo se desarrolla el software a nivel profesional: la **Programación Orientada a Objetos (POO)**.

En lugar de tener los datos (el importe, el concepto) separados de las acciones (registrar, calcular IVA), la POO nos permite agruparlo todo en una sola entidad lógica. Vamos a descubrir cómo aplicar este paradigma a la economía de tu día a día.



Imagen de elaboración propia generada con ChatGPT <<https://chatgpt.org>> .
Programación orientada a Objetos (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

2. ¿Por qué la programación evolucionó hacia los objetos?

Imagina que trabajas en un banco y tienes que programar el sistema que gestiona las cuentas de todos los clientes. Con programación estructurada tendrías miles de variables sueltas y funciones que operan sobre ellas. ¿Cómo sabes qué función pertenece a qué cuenta? ¿Cómo evitas que una función modifique por error los datos de otro cliente?

La respuesta que encontraron los programadores en los años 80 fue inspirarse en cómo funciona el **mundo real**. En el mundo real no pensamos en variables y funciones: pensamos en **objetos**. Un coche tiene características (color, marca, velocidad) y comportamientos (arrancar, frenar, acelerar). Una cuenta bancaria tiene características (titular, saldo, número) y comportamientos (ingresar, retirar, consultar saldo). Una factura tiene características (importe, fecha, concepto) y comportamientos (calcular IVA, emitir, pagar).

La Programación Orientada a Objetos (POO) traslada exactamente esta forma de pensar al código.

<https://www.youtube.com/embed/IYpJhm2BRB0>
<<https://www.youtube.com/embed/IYpJhm2BRB0>>

<<https://www.youtube.com/embed/IYpJhm2BRB0>>

Error de configuración del reproductor de vídeo

Error 153

Ver vídeo en YouTube

Más información

Error 153

Vídeo de Diego López Dev <<https://www.youtube.com/@DiegoLopezDev>> . Programación Orientada a Objetos (POO) explicada fácil y divertido <<https://www.youtube.com/watch?v=IYpJhm2BRB0>> (Licencia de YouTube estándar <<https://www.youtube.com/static?template=terms>>)

3. POO en la vida real

Algunos ejemplos concretos del uso de la POO en la vida real.

Industrias y aplicaciones

La Programación Orientada a Objetos está detrás de prácticamente todo el software que usas en tu vida diaria. Cuando abres una app de mensajería, cada mensaje es un objeto con sus atributos (texto, fecha, remitente) y sus métodos (enviar, eliminar, reenviar). Cuando usas una tienda online, cada producto es un objeto con su precio, stock y categoría.

Las grandes empresas tecnológicas como Google, Amazon, Netflix o Spotify construyen sus sistemas con POO porque les permite organizar equipos de cientos de programadores trabajando sobre el mismo proyecto sin pisarse: cada equipo se encarga de sus propias clases y objetos, que luego se integran con el resto del sistema.



Imagen de elaboración propia generada con Gemini <<https://gemini.google.com/>> .
POO en la industria (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)

Bancas y Finanzas

Cualquier aplicación bancaria usa POO. Una clase *CuentaBancaria* define los atributos comunes (titular, saldo, número) y los métodos básicos (ingresar, retirar, consultar). De ella heredan *CuentaCorriente* y *CuentaAhorro*, añadiendo cada una sus propias características sin repetir código. Exactamente lo mismo que harás con tu *Gasto*.



Imagen de Tech Daily en Unsplash <<https://unsplash.com/es/@techdailyca>> . Banca Online <<https://unsplash.com/es/fotos/persona-sosteniendo-un-telefono-inteligente-samsung-android-negro-GdRvIi8mWzE>> (Licencia de Unsplash <<https://unsplash.com/es/licencia>>)

Videojuegos

En un videojuego, la creación de diferentes opciones de jugador, sus diferentes roles, los skins de Fortnite o los diversos enemigos son un ejemplo sencillo de POO. Una clase *Personaje* define lo que tienen en común todos los personajes y de ella heredan *Heroe*, *Enemigo* o *NPC* con sus propias características.



Imagen de Edge Tung en Unsplash <<https://unsplash.com/es/@se7ened>> .
Videojuegos <<https://unsplash.com/es/fotos/cabeza-gigante-de-mario-emergiendo-de-una-tuberia-verde-ZPTHesvkYy4>> (Licencia de Unsplash <<https://unsplash.com/es/licencia>>)

Interfaces Gráficas

Ya lo viste en el bloque de bibliotecas: en *tkinter* cada elemento de la interfaz es un objeto. *Button*, *Label* y *Entry* son clases que heredan de una clase base común *Widget*. Cuando llamas a `btn = tk.Button(ventana, text="Registrar")` estás creando un objeto de la clase *Button*.



Imagen de elaboración propia. Interfaz de usuario (CC BY-NC-SA
<<http://creativecommons.org/licenses/>>)

4. Modela un objeto

Acabas de descubrir que la Programación Orientada a Objetos está basada en cómo concebimos la realidad. Cualquier objeto del mundo real tiene características que lo describen y acciones que puede realizar. En POO llamamos a estas características atributos y a las acciones métodos.

Fíjate en un objeto que usas en tu día a día: una tarjeta de crédito. Tiene un número, un titular, una fecha de caducidad... y puede realizar acciones como pagar, consultar el saldo o bloquearla.

Observa los elementos de una tarjeta de crédito y clasifica cada uno según sea un atributo (característica estática que describe el objeto) o un método (acción dinámica que puede realizar el objeto)



Imagen de Jjuni en Pixabay, Tarjeta de crédito (Licencia Pixabay)

Ambos tienen que estar indentados por 4 espacios dentro de la clase.

Ejemplo del Gestor:

```
class Gasto:

    # Atributos: características que tendrá cada gasto
    importe = 0.0
    categoria = ""
    fecha = ""

    # Métodos: comportamientos del objeto
    def mostrar(self):
        print(f"{self.fecha} | {self.categoria:<12} | {self.importe:.2f} €")

    def es_elevado(self):
        return self.importe > 100
```

Constructor `__init__` y `self`

El código anterior funciona, pero tiene un problema: todos los objetos *Gasto* nacerían con los mismos valores por defecto (*importe* = 0.0, *categoria* = ""). Para que cada objeto nazca ya con sus propios datos, Python usa un método especial llamado constructor: `__init__`.

El constructor se ejecuta automáticamente cada vez que creas un nuevo objeto y es donde se asignan los valores iniciales de sus atributos:

```
class Gasto:

    # Constructor: se ejecuta al crear cada objeto
    def __init__(self, importe, categoria, fecha):
        self.importe = importe # Atributo del objeto
        self.categoria = categoria # Atributo del objeto
        self.fecha = fecha # Atributo del objeto

    # Métodos: comportamientos del objeto
    def mostrar(self):
        print(f"{self.fecha} | {self.categoria:<12} | {self.importe:.2f} €")

    def es_elevado(self):
        return self.importe > 100
```

¿Qué es `self`?

`self` hace referencia al propio objeto que se está creando. Es obligatorio como primer parámetro en todos los métodos de una clase, aunque no lo pasas tú al llamarlos: Python lo gestiona automáticamente.

Piensa en `self` como "yo mismo": cuando el objeto `gasto1` llama a `mostrar()`, `self` es `gasto1`. Cuando lo llama `gasto2`, `self` es `gasto2`.

Crear objetos: instanciación

Una vez definida la clase, para usar los objetos es necesario crearlos, para ello utilizamos el nombre de la clase con los parámetros, entre paréntesis, si fueran necesarios:

```
# Creamos tres objetos de la clase Gasto
gasto1 = Gasto(45.50, "comida", "01/11/2024")
gasto2 = Gasto(120.00, "casa", "05/11/2024")
gasto3 = Gasto(18.75, "transporte", "08/11/2024")

# Accedemos a sus atributos con la notación punto
print(gasto1.importe) # 45.5
print(gasto2.categoria) # casa

# Llamamos a sus métodos con la notación punto
gasto1.mostrar() # 01/11/2024 | comida | 45.50 €
gasto2.mostrar() # 05/11/2024 | casa | 120.00 €
print(gasto3.es_elevado()) # False
```

Guardar objetos en lista

En tu Gestor de Gastos querrás registrar muchos gastos. La forma natural es guardar los objetos en una lista:

```
# Lista de objetos Gasto
gastos = []

# Añadimos gastos
gastos.append(Gasto(45.50, "comida", "01/11/2024"))
gastos.append(Gasto(120.00, "casa", "05/11/2024"))
gastos.append(Gasto(18.75, "transporte", "08/11/2024"))
```

```
# Recorremos y mostramos todos
print(" Listado de gastos:")
for g in gastos:
    g.mostrar()

# Calculamos el total
total = sum(g.importe for g in gastos)
print(f"\n Total: {total:.2f} €")
```

Trabajando con objetos

Ya sabes crear objetos. Ahora veamos cómo trabajar con ellos de forma más completa: acceder a sus datos, modificarlos y hacer que se representen de forma inteligente.

Acceder y modificar atributos

Con la notación punto puedes leer y modificar los atributos de cualquier objeto directamente:

```
gasto1 = Gasto(45.50, "comida", "01/11/2024")

# Leer atributos
print(gasto1.importe)      # 45.5
print(gasto1.categoria)    # comida

# Modificar un atributo
gasto1.importe = 50.00
print(gasto1.importe)      # 50.0

# Usar atributos en expresiones
presupuesto = 500
if gasto1.importe > presupuesto * 0.10:
    print(" Gasto considerable")
```

Método str

Cuando intentas imprimir un objeto directamente, Python muestra algo poco útil como `<__main__.Gasto object at 0x...>`. Para solucionarlo existe el método especial `__str__`: define cómo se representa el objeto como texto:

```
class Gasto:

    def __init__(self, importe, categoria, fecha):
        self.importe = importe
        self.categoria = categoria
        self.fecha = fecha

    def __str__(self):
        return f"{self.fecha} | {self.categoria:<12} | {self.importe:.2f} €"

    def es_elevado(self):
        return self.importe > 100

# Ahora print() muestra algo útil
gasto1 = Gasto(45.50, "comida", "01/11/2024")
print(gasto1)      # 01/11/2024 | comida          | 45.50 €
```

`__str__` se llama automáticamente cuando usas `print(objeto)` o `str(objeto)`. Es el equivalente a decirle al objeto: "cuando alguien te pida que te muestres como texto, hazlo así".

2. Revisamos lo aprendido

Elija la respuesta correcta

7

00:00



[Pulse aquí para jugar](#)

10.2 Herencia y Biblioteca de clases

1. Herencia

Ya sabes crear clases y objetos. Ahora imagina que en tu Gestor necesitas diferenciar entre dos tipos de gastos: los gastos variables (compras, ocio, imprevistos) y los gastos fijos (alquiler, suscripciones, seguros). Ambos tienen importe, categoría y fecha, pero los fijos además tienen un día de cobro mensual y se repiten automáticamente.

Antes de pasar al ejemplo de nuestro Gestor, vamos a ver el clásico ejemplo del vehículo, para entender el concepto de herencia.

Imagina que tienes que modelar distintos tipos de vehículos. Todos comparten **características** comunes (marca, velocidad, número de ruedas) y **comportamientos** básicos (arrancar, parar). Pero un coche y una moto también tienen sus propias particularidades.

Sin herencia tendrías que crear dos clases completamente distintas repitiendo todo el código común. Con **herencia**, la clase hija (**subclase**) reutiliza todo lo de la clase padre (**superclase**), heredando sus atributos y métodos y solo añade lo que la diferencia.



Imagen de elaboración propia generada con Claude <<https://claude.ai/>> . Herencia en Python (CC BY-NC-SA <<http://creativecommons.org/licenses/by-nc-sa/4.0/>>)



`super()` permite llamar a los métodos de la clase padre desde la clase hija. Su uso más importante es en el constructor:

```
# En Coche.__init__:

super().__init__(marca, velocidad_max, color)

# Es equivalente a escribir:
Vehiculo.__init__(self, marca, velocidad_max, color)
```

Fíjate en la diferencia entre hacerlo **con** y **sin** `super()`:

```
# ❌ Sin super(): repetimos el código del padre
class Coche(Vehiculo):
    def __init__(self, marca, velocidad_max, color, num_puertas):
        self.marca = marca # Repetido del padre
```

```

        self.velocidad_max = velocidad_max # Repetido del padre
        self.color = color # Repetido del padre
        self.num_puertas = num_puertas # Propio de Coche

# ✅ Con super(): reutilizamos el constructor del padre
class Coche(Vehiculo):
    def __init__(self, marca, velocidad_max, color, num_puertas):
        super().__init__(marca, velocidad_max, color) # Una sola línea
        self.num_puertas = num_puertas # Propio de Coche

```

Usando `super().__init__()` evitas repetir en la clase hija el código del constructor del padre. Si mañana añades un atributo nuevo a `Vehiculo`, como matrícula, todas las clases hijas lo heredarán automáticamente sin tocar su código.

Veamos ahora como sería el código de la clase `Vehiculo` y las clases hijas (`coche` y `moto`).

```

# — CLASE PADRE —————
class Vehiculo:

    def __init__(self, marca, velocidad_max):
        self.marca = marca
        self.velocidad_max = velocidad_max

    def __str__(self):
        return f"{self.marca} (máx. {self.velocidad_max} km/h)"

    def arrancar(self):
        print(f" {self.marca} arrancando...")

```

```

# — CLASE HIJA: Coche hereda de Vehiculo —————
class Coche(Vehiculo):

    def __init__(self, marca, velocidad_max, num_puertas):
        super().__init__(marca, velocidad_max) # Llama al padre
        self.num_puertas = num_puertas # Atributo propio

    def __str__(self):
        return f"{super().__str__()} · {self.num_puertas} puertas"

    def abrir_maletero(self):
        print(f"Maletero del {self.marca} abierto")

```

```

# — CLASE HIJA: Moto hereda de Vehiculo —————
class Moto(Vehiculo):

    def __init__(self, marca, velocidad_max, con_sidecar):
        super().__init__(marca, velocidad_max) # Llama al padre
        self.con_sidecar = con_sidecar # Atributo propio

    def __str__(self):
        sidecar = "con sidecar" if self.con_sidecar else "sin sidecar"
        return f"{super().__str__()} · {sidecar}"

    def hacer_caballito(self):
        print(f" ¡{self.marca} haciendo caballito!")

```



Prueba a continuación el siguiente código y comprueba cómo funciona la herencia:

```

# Creamos objetos de cada clase
v = Vehículo("Genérico", 200)
c = Coche("Seat Ibiza", 180, 5)
m = Moto("Honda CBR", 240, False)

# Todos heredan arrancar() del padre
v.arrancar() # Genérico arrancando...
c.arrancar() # Seat Ibiza arrancando...
m.arrancar() # Honda CBR arrancando...

# Cada uno muestra su propio __str__
print(v) # Genérico (máx. 200 km/h)
print(c) # Seat Ibiza (máx. 180 km/h) · 5 puertas
print(m) # Honda CBR (máx. 240 km/h) · sin sidecar

# Métodos propios de cada clase hija
c.abrir_maletero() # Maletero del Seat Ibiza abierto
m.hacer_caballito() # ¡Honda CBR haciendo caballito!

# isinstance() distingue entre tipos
vehiculos = [v, c, m]
for veh in vehiculos:
    if isinstance(veh, Coche):
        print(f"{veh.marca} es un Coche con {veh.num_puertas} puertas")
    elif isinstance(veh, Moto):
        print(f"{veh.marca} es una Moto")
    else:
        print(f"{veh.marca} es un Vehículo genérico")

```

La salida del terminal tiene que ser similar a la siguiente:

```

PS C:\Users\usuario\AppData\Local\Programs\Microsoft VS Code>
Genérico arrancando...
Seat Ibiza arrancando...
Honda CBR arrancando...
Genérico (máx. 200 km/h)
Seat Ibiza (máx. 180 km/h) · 5 puertas
Honda CBR (máx. 240 km/h) · sin sidecar
Maletero del Seat Ibiza abierto
¡Honda CBR haciendo caballito!
Genérico es un Vehículo genérico
Seat Ibiza es un Coche con 5 puertas
Honda CBR es una Moto

```

Imagen de elaboración propia. Salida terminal prueba herencia (CC BY-NC-SA)
<http://creativecommons.org/licenses/by-nc-sa/4.0/>

2. Relaciona

Relaciona el concepto con su descripción

6

Comprobar



Clase hija

Herencia

super().__init__()

Método sobrescrito

Clase Padre

Método del padre redefinido en la clase hija con nuevo comportamiento

Llama al constructor del padre desde la clase hija

Clase que hereda de otra y añade sus propias características

Mecanismo que permite reutilizar el código de una clase en otra

Comprueba si un objeto es instancia de una clase concreta

isinstance(obj, Clase)

Clase de la que heredan otras clases.
Define lo común

3. Biblioteca de clases

A lo largo del curso ya has usado bibliotecas de clases sin saberlo. Cuando escribes `from datetime import date` estás importando una clase llamada `date`. Cuando creas `hoy = date.today()` estás creando un objeto de esa clase:

```
POO.py X
1 from datetime import date, datetime
2
3 # date y datetime son clases de la biblioteca datetime
4 hoy = date.today() # objeto de la clase date
5 ahora = datetime.now() # objeto de la clase datetime
6
7 # Sus métodos
8 print(hoy.strftime("%d/%m/%Y")) # método del objeto hoy
9 print(ahora.hour) # atributo del objeto ahora
```

Imagen de elaboración propia. Bibliotecas de clase (CC BY-NC-SA
<<http://creativecommons.org/licenses/>>)

También `tk.Button`, `tk.Label` y `tk.Entry` de `tkinter` son clases que heredan de una clase base `Widget`. Cada vez que creabas un botón en tu interfaz, estabas instanciando la clase `Button`.

4. ¿Qué ocurre en este código?

Elige la respuesta correcta

3 0 0 0

⌚ 00:00



[Pulse aquí para jugar](#)

5. Del vehículo al Gestor de Gastos

A lo largo de este bloque has aprendido los conceptos de la Programación Orientada a Objetos usando el ejemplo del vehículo. Ahora es el momento de aplicar todo lo aprendido a tu propio proyecto: el Gestor de Gastos Personales.

En esta actividad vas a reescribir tu Gestor usando dos clases que modelan los tipos de gastos que puede registrar la aplicación.

Las clases que debes crear:

📌 Clase Gasto — clase padre.

Representa un gasto variable del mes. Debe tener:

- **Atributos:** importe, categoría y fecha (asignada automáticamente con `datetime`).
- **Método `__init__`:** recibe importe y categoría como parámetros.
- **Método `__str__`:** devuelve una línea con la fecha, categoría e importe formateado.
- **Método `es_elevado()`:** devuelve `True` si el importe supera los 100 €.
- **Método `es_valido()`:** devuelve `True` si la categoría está en la tupla `CATEGORIAS`.

📌 Clase GastoFijo — clase hija que hereda de Gasto.

Representa un gasto que se repite cada mes (alquiler, suscripciones...). Debe tener:

- **Atributo propio:** `dia_cobro` (día del mes en que se cobra).
- **Método `__init__`:** recibe importe, categoría y `dia_cobro`. Usa `super().__init__()` para reutilizar el constructor del padre.
- **Método `__str__`:** amplía el del padre añadiendo `[Fijo · día X]`.
- **Método `aviso_cobro()`:** muestra un mensaje indicando qué día del mes se cobra ese gasto.

Lo que debe hacer el programa

Una vez definidas las clases, escribe el programa principal que:

1. Defina la tupla `CATEGORIAS` con: comida, transporte, ocio, casa, salud.
2. Cree al menos 3 gastos fijos usando la clase `GastoFijo` (por ejemplo: alquiler, Netflix, seguro).
3. Cree al menos 3 gastos variables usando la clase `Gasto` (por ejemplo: comida, ocio, transporte).
4. Guarde todos los gastos en una sola lista mezclando ambos tipos.
5. Muestre el listado completo de gastos usando `print(g)` para cada uno, añadiendo 🚩 si el gasto es elevado.
6. Muestre los avisos de cobro solo para los gastos fijos usando `isinstance()`.
7. Calcule y muestre el total, desglosado en gastos fijos y variables.

Así debe verse la salida de tu programa:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\usuario\AppData\Local\Programs\Microsoft VS Code>
Listado completo de gastos:

30/08/2026 | casa      | 650.00 € [Fijo · día 1] 🚩
30/08/2026 | ocio      | 15.99 € [Fijo · día 15]
30/08/2026 | transporte | 89.00 € [Fijo · día 5]
30/08/2026 | comida    | 45.50 €
30/08/2026 | ocio      | 18.75 €
30/08/2026 | transporte | 32.00 €

📅 Próximos cobros fijos:
📅 'casa' se cobra el día 1 de cada mes
📅 'ocio' se cobra el día 15 de cada mes
📅 'transporte' se cobra el día 5 de cada mes

💰 Total gastos: 851.24 €
➔ Fijos: 754.99 €
➔ Variables: 96.25 €
```

Imagen de elaboración propia. Salida Terminal Herencia Gastos Fijos (CC BY-NC-SA
<<http://creativecommons.org/licenses/>>)



- 🔹 Usa `import datetime` para asignar la fecha automáticamente en `__init__`.
- 🔹 En `GastoFijo.__init__` llama a `super().__init__(importe, categoria)` antes de asignar `dia_cobro`.
- 🔹 En `GastoFijo.__str__` usa `super().__str__()` para no repetir el código del padre.
- 🔹 Usa `isinstance(g, GastoFijo)` dentro del bucle para distinguir los tipos de gasto.

```
import datetime

CATEGORIAS = ("comida", "transporte", "ocio", "casa", "salud")

class Gasto:
```

```

"""Representa un gasto variable."""

def __init__(self, importe, categoria):
    self.importe = float(importe)
    self.categoria = categoria.lower().strip()
    self.fecha = datetime.date.today().strftime("%d/%m/%Y")

def __str__(self):
    return f"{self.fecha} | {self.categoria:<12} | {self.importe:.2f} €"

def es_elevado(self):
    return self.importe > 100

def es_valido(self):
    return self.categoria in CATEGORIAS

class GastoFijo(Gasto):
    """Representa un gasto fijo mensual. Hereda de Gasto."""

    def __init__(self, importe, categoria, dia_cobro):
        super().__init__(importe, categoria)
        self.dia_cobro = dia_cobro

    def __str__(self):
        return f"{super().__str__()} [Fijo · día {self.dia_cobro}]"

    def aviso_cobro(self):
        print(f"🔔 '{self.categoria}' se cobra el día {self.dia_cobro} de cada mes")

# — Programa principal —————
gastos = []

# Gastos fijos mensuales
gastos.append(GastoFijo(650.00, "casa", 1))
gastos.append(GastoFijo(15.99, "ocio", 15))
gastos.append(GastoFijo(89.00, "transporte", 5))

# Gastos variables
gastos.append(Gasto(45.50, "comida"))
gastos.append(Gasto(18.75, "ocio"))
gastos.append(Gasto(32.00, "transporte"))

# Mostrar todos
print("📋 Listado completo de gastos:")
print("-" * 52)
for g in gastos:
    alerta = " ⚠️ " if g.es_elevado() else ""
    print(f"{g}{alerta}")

# Avisos de cobro solo para fijos
print("\n🔔 Próximos cobros fijos:")
for g in gastos:
    if isinstance(g, GastoFijo):
        g.aviso_cobro()

# Totales desglosados
total = sum(g.importe for g in gastos)
fijos = sum(g.importe for g in gastos if isinstance(g, GastoFijo))
variables = total - fijos

print(f"\n🔥 Total gastos: {total:.2f} €")
print(f" → Fijos: {fijos:.2f} €")
print(f" → Variables: {variables:.2f} €")

```

¿Tu programa no funciona? Comprueba esto:

- 🔦 ¿La clase GastoFijo incluye (Gasto) en su definición?
- 🔦 ¿Usas super().__init__() en el constructor de GastoFijo?
- 🔦 ¿Usas isinstance(g, GastoFijo) para distinguir los tipos?
- 🔦 ¿El __str__ de GastoFijo llama a super().__str__()?

📌 Reto extra

Crea una tercera clase GastoExtraordinario(Gasto) para gastos imprevistos importantes con un atributo motivo y un método generar_informe() que muestre todos sus datos detalladamente.



6. Importante

Clase y Objeto

Una clase es el molde que define la estructura: qué atributos tiene y qué puede hacer. Un objeto es una instancia concreta creada a partir de ese molde, con sus propios valores.

```
class Gasto:           # Molde
    pass

g1 = Gasto()           # Objeto concreto
```

La clase se define una sola vez. Los objetos se crean tantas veces como necesites.

El constructor `__init__` y `self`

`__init__` se ejecuta automáticamente al crear un objeto y asigna los valores iniciales de sus atributos. `self` hace referencia al propio objeto que se está creando.

```
class Gasto:
    def __init__(self, importe, categoria):
        self.importe = importe # Atributo del objeto
        self.categoria = categoria # Atributo del objeto
```

`self` es obligatorio como primer parámetro en todos los métodos, aunque Python lo gestiona automáticamente al llamarlos.

Atributos y métodos

Los **atributos** son las características del objeto (qué tiene). Los **métodos** son sus comportamientos (qué hace). Se accede a ambos con la **notación punto**.

```
g1 = Gasto(45.50, "comida")

# Acceder a atributos
print(g1.importe) # 45.5

# Llamar a métodos
g1.mostrar() # Ejecuta el método
```

El método `str`

Define cómo se representa el objeto cuando usas `print()`. Sin él, Python muestra una dirección de memoria poco útil.

```
def __str__(self):
    return f"{self.fecha} | {self.categoria} | {self.importe:.2f} €"

print(g1) # 06/11/2024 | comida | 45.50 €
```

Herencia

Una clase hija hereda todos los atributos y métodos de la clase padre y añade los suyos propios. Se indica poniendo la clase padre entre paréntesis.

```
class GastoFijo(Gasto): # Hereda de Gasto
    def __init__(self, importe, categoria, dia_cobro):
        super().__init__(importe, categoria) # Constructor del padre
        self.dia_cobro = dia_cobro # Atributo propio
```

`super().__init__()` reutiliza el constructor del padre sin repetir código. Si el padre cambia, la clase hija se actualiza automáticamente.

`isinstance()`

Cuando tienes una lista con objetos de distintas clases, `isinstance()` te permite identificar de qué tipo es cada uno.

```
for g in gastos:
    if isinstance(g, GastoFijo):
        g.aviso_cobro() # Solo los gastos fijos
```

11. Resumen

1. Índice de la SdA

2. Importante

Antes de comenzar a construir tu Gestor de Gastos Personales, asegúrate de que dominas los contenidos fundamentales de esta SdA. Repasa cada apartado y, si alguno no te queda claro, vuelve al bloque correspondiente antes de continuar.

El idioma de las máquinas

- Los lenguajes de alto nivel están diseñados para el programador; los de bajo nivel, para el ordenador.
- Los lenguajes compilados traducen todo el código de una vez; los interpretados (como Python) lo hacen línea a línea.
- Todo programa sigue la estructura Entrada → Procesamiento → Salida.
- Python es un lenguaje interpretado, gratuito, sencillo y muy demandado en el mercado laboral.
- El entorno de trabajo será VS Code.

Los cimientos del código

- Una variable es un contenedor con nombre que almacena un dato. Una constante no cambia (se escribe en MAYÚSCULAS por convenio).
- Los tipos de datos básicos son: *int* (entero), *float* (decimal), *str* (texto) y *bool* (verdadero/falso).
- type()* permite conocer el tipo de dato de una variable; *int()*, *float()* y *str()* convierten entre tipos.
- Los operadores aritméticos (+, -, *, /, //, %) realizan cálculos matemáticos.
- Los operadores de asignación (=, +=, -=) asignan o actualizan el valor de una variable.
- Los operadores de comparación (==, !=, >, <, >=, <=) devuelven *True* o *False*.
- Los operadores lógicos (*and*, *or*, *not*) combinan condiciones.
- input()* recoge datos del usuario siempre como texto; *print()* muestra resultados; las *f-strings* formatean la salida.

El programa decide y repite

- La estructura *if / elif / else* ejecuta bloques de código según se cumplan condiciones.
- La indentación (4 espacios) es obligatoria en Python para definir los bloques de código.
- El bucle *for* repite un bloque un número determinado de veces; se usa con *range()* o para recorrer colecciones.
- El bucle *while* repite mientras se cumpla una condición; puede usarse como contador o acumulador.
- break* sale del bucle inmediatamente; *continue* salta a la siguiente iteración.
- Un bucle infinito ocurre cuando la condición nunca se vuelve *False*; se controla con *break* o actualizando la variable.

Organiza tus datos

- Una lista (`[]`) es ordenada, modificable y permite duplicados → ideal para almacenar los importes del mes.
- Una tupla (`()`) es ordenada e inmutable → ideal para las categorías fijas del sistema.
- Un diccionario (`{clave: valor}`) accede a los datos por nombre → ideal para el total por categoría.
- Un conjunto (`{}`) no admite duplicados ni orden → ideal para obtener las categorías únicas usadas.
- Métodos principales de lista: *append()*, *insert()*, *remove()*, *pop()*, *sort()*, *reverse()*, *count()*, *extend()*.

Divide y reutiliza

- Una función se define con *def*, recibe parámetros y puede devolver un resultado con *return*.

- ◆ El principio DRY (Don't Repeat Yourself): escribe el código una vez y reutilízalo todas las veces que necesites.
- ◆ Las variables locales solo existen dentro de la función; las globales son accesibles desde todo el programa.
- ◆ Las bibliotecas estándar (*datetime*, *csv*, *json*, *os*, *tkinter*) vienen incluidas con Python.
- ◆ Las bibliotecas de terceros (*matplotlib*) necesitan instalarse con *pip install*.
- ◆ *import* biblioteca importa el módulo completo; *from biblioteca import función* importa solo lo necesario.

La memoria a largo plazo

- ◆ Abre siempre los archivos con *with open(...) as f* — se cierra automáticamente aunque ocurra un error.
- ◆ Modos de apertura: "r" (leer), "w" (escribir desde cero), "a" (añadir), "x" (crear nuevo).
- ◆ Usa *os.path.exists()* antes de abrir un archivo para evitar errores en la primera ejecución.
- ◆ *json.dump()* guarda listas y diccionarios; *json.load()* los recupera. Añade siempre *ensure_ascii=False*.
- ◆ El bloque *try / except* captura errores y evita que el programa se cierre bruscamente.
- ◆ Captura siempre al menos *FileNotFoundError* y *json.JSONDecodeError* al trabajar con archivos

El mundo en objetos

- ◆ Una clase es el molde que define atributos y métodos; un objeto es una instancia concreta de esa clase.
- ◆ El constructor *__init__* se ejecuta automáticamente al crear un objeto y asigna sus valores iniciales.
- ◆ *self* hace referencia al propio objeto — es obligatorio como primer parámetro en todos los métodos.
- ◆ El método *__str__* define cómo se muestra el objeto al usar *print()*.
- ◆ La herencia permite que una clase hija reutilice los atributos y métodos de la clase padre.
- ◆ *super().__init__()* llama al constructor del padre desde la clase hija sin repetir código.
- ◆ *isinstance(objeto, Clase)* comprueba de qué tipo es un objeto — útil cuando mezclas tipos en una lista.

¿Estás listo para el reto?

Antes de pasar a construir tu Gestor de Gastos comprueba que:

- ✓ Sabes crear variables, listas, tuplas y diccionarios en Python.
- ✓ Puedes escribir un menú interactivo con *while* y *if/elif/else*.
- ✓ Sabes dividir el código en funciones con *def*.
- ✓ Entiendes cómo guardar y cargar datos con *json* y *os*.
- ✓ Conoces la diferencia entre una clase padre y una clase hija.

Si has marcado todos, ¡estás más que preparado! Pasa al siguiente apartado y comienza a construir tu Gestor de Gastos Personales.

3. Vídeo resumen

A continuación tienes un vídeo resumen de la situación de aprendizaje:

<https://www.youtube.com/embed/MHjRWYwMT1w>
<<https://www.youtube.com/embed/MHjRWYwMT1w>>

<<https://www.youtube.com/embed/MHjRWYwMT1w>>

Error de configuración del reproductor de vídeo

Error 153

Ver vídeo en YouTube

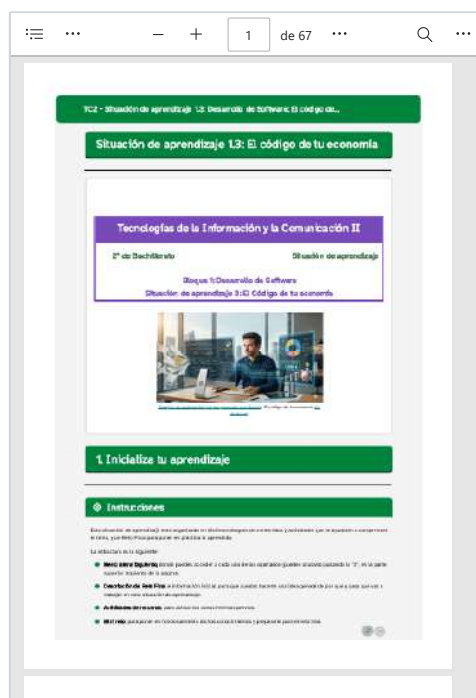
Más información

Error 153

Video de Educación Permanente <<https://www.youtube.com/@Educaci%C3%B3nPermanente-c1w>> . ES Saltar navegación Crear Imagen del avatar Generar video resumen de una SdA con Gemini Notebook <<https://youtu.be/MHjRWYwMT1w>> (CC BY <<http://creativecommons.org/licenses/by/>>)

Imprimible

Descarga [aquí](#) la versión imprimible.



Material de elaboración propia. Imprimible de la situación de aprendizaje
X.X (CC BY-NC-SA)

Créditos

Detalle y autoría

Título	Situación de aprendizaje 1.3: El código de tu economía
Enseñanza y nivel	2º de Bachillerato
Descripción	REA de la asignatura Tecnologías de la Información y la Comunicación II para 2º de Bachillerato
Elaboración del recurso	María Teresa Zamorano Osuna
Revisión del recurso	Fermin Martínez Revelles
Coordinación	Alejandro López Vega
Supervisión	José María Reyes Cazalla
Organización	Consejería de Educación. Junta de Andalucía.
Licencia	Licencia Creative Commons Reconocimiento No comercial Compartir igual 4.0 < http://creativecommons.org/licenses/by-nc-sa/4.0/ >

Este contenido fue creado con eXeLearning <<http://exelearning.net/>> , el editor libre y de fuente abierta diseñado para crear recursos educativos.

Historial de versiones

Elaborado por:	Servicio de Educación Permanente. Consejería de Educación. Junta de Andalucía.		
Versión:	01	Fecha de publicación:	Septiembre 2026 Primera versión

Aviso legal

<https://www.juntadeandalucia.es/educacion/permanente/materiales/index.php?aviso#space>



Buscar...

